

AI/LLM 与向量技术教学指南

从基础理论到工程实践

技术教学文档

September 13, 2025

Contents

1 引言	3
1.1 学习目标	3
2 大语言模型基础	3
2.1 什么是大语言模型	3
2.1.1 核心概念	3
2.1.2 数学基础	3
2.2 注意力机制原理	3
2.2.1 自注意力公式	4
3 提示工程 (Prompt Engineering)	4
3.1 基本原理	5
3.2 提示类型分类	5
3.3 实践示例	5
3.3.1 零样本提示 (Zero-shot)	5
3.3.2 少样本提示 (Few-shot)	5
3.3.3 思维链提示 (Chain-of-Thought)	6
3.4 上下文管理技术	6
3.4.1 Token 计算与管理	6
4 Model Context Protocol (MCP)	8
4.1 MCP 架构原理	9
4.2 MCP 核心组件	9
4.3 MCP 服务器实现	9
5 多智能体系统	14
5.1 核心概念	14
5.2 AutoGen 框架	14
5.2.1 基本架构	14
5.2.2 AutoGen 实现示例	14
5.3 CrewAI 任务协作模式	18

6 向量数据库技术基础	23
6.1 向量的数学基础	24
6.1.1 向量表示	24
6.1.2 相似度计算	24
6.2 向量索引算法	24
6.2.1 HNSW 算法原理	24
6.2.2 HNSW 参数优化	25
6.3 Qdrant 实战应用	25
7 检索增强生成 (RAG) 系统	35
7.1 RAG 系统架构	36
7.2 RAG 核心数学原理	36
7.2.1 检索相关性计算	36
7.2.2 概率生成模型	36
7.3 高级 RAG 实现	37
8 性能优化与评估	48
8.1 系统性能指标	48
8.2 优化策略	49
8.2.1 索引优化	49
8.2.2 缓存策略	49
9 实战练习与案例分析	56
9.1 综合案例：智能文档问答系统	56
10 总结与进阶学习路径	63
10.1 技术要点回顾	63
10.2 进阶学习建议	64
10.3 推荐资源	64
11 附录：常用公式和算法	65
11.1 数学公式速查	65
11.1.1 向量相似度计算	65
11.1.2 注意力机制	65
11.2 性能评估指标	65
11.2.1 检索质量指标	65
11.2.2 系统性能指标	66
11.3 算法复杂度分析	66
12 实用工具和脚本	66
12.1 性能测试脚本	66
12.2 系统监控仪表盘	71
13 结语	76

1 引言

本文档旨在为初学者提供 AI/LLM 专业技术和数据向量技术的全面教学指南。我们将从基础概念开始，逐步深入到高级应用和工程实践。

1.1 学习目标

- 理解大语言模型 (LLM) 的工作原理和应用场景
- 掌握提示工程和上下文管理技术
- 学会多智能体系统的设计和实现
- 深入了解向量数据库和检索技术
- 能够构建完整的 RAG(检索增强生成) 系统

2 大语言模型基础

2.1 什么是大语言模型

大语言模型 (Large Language Model, LLM) 是一种基于深度学习的自然语言处理模型，通过在大量文本数据上训练，学会理解和生成人类语言。

2.1.1 核心概念

- **Transformer 架构**: 基于注意力机制的神经网络架构
- **自回归生成**: 根据前面的词预测下一个词
- **上下文窗口**: 模型一次能处理的最大 token 数量
- **Token**: 文本的最小处理单位（可能是词、子词或字符）

2.1.2 数学基础

LLM 的核心是计算下一个 token 的条件概率：

$$P(x_{t+1}|x_1, x_2, \dots, x_t) = \text{softmax}(f(x_1, x_2, \dots, x_t)) \quad (1)$$

其中：

- x_i 表示第 i 个 token
- $f(\cdot)$ 是 transformer 网络函数
- softmax 函数将输出转换为概率分布

2.2 注意力机制原理

注意力机制是 Transformer 的核心，用于计算序列中各位置之间的关系。

2.2.1 自注意力公式

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (2)$$

$$\text{其中: } Q = XW_Q \quad (3)$$

$$K = XW_K \quad (4)$$

$$V = XW_V \quad (5)$$

参数说明:

- Q : 查询矩阵 (Query)
- K : 键矩阵 (Key)
- V : 值矩阵 (Value)
- d_k : 键向量的维度
- W_Q, W_K, W_V : 可学习的权重矩阵

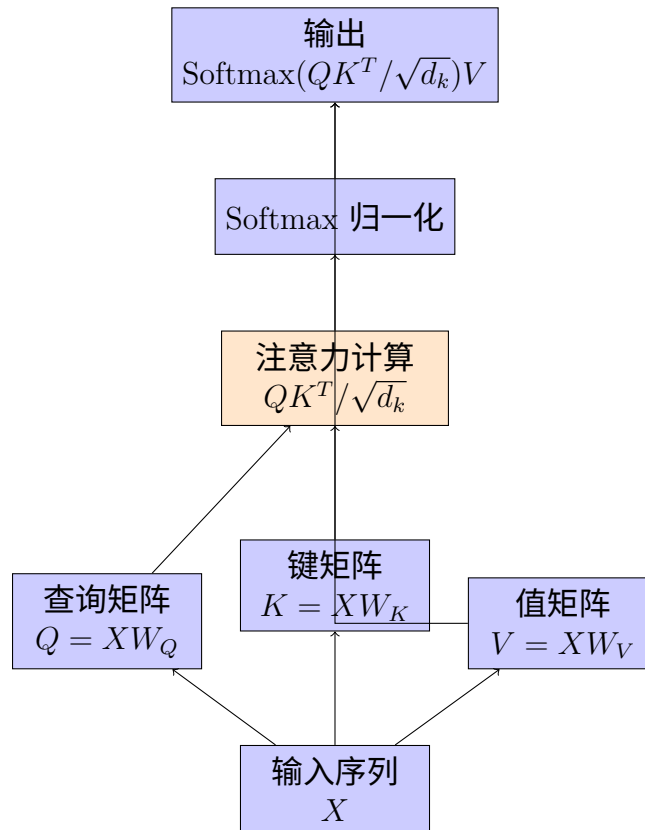


Figure 1: Transformer 注意力机制流程图

3 提示工程 (Prompt Engineering)

提示工程是与 LLM 交互的艺术和科学，通过精心设计输入提示来获得期望的输出结果。

3.1 基本原理

提示工程基于以下原理：

1. **条件生成**: LLM 根据给定条件生成文本
2. **上下文学习**: 在提示中提供示例来指导行为
3. **角色扮演**: 为模型分配特定身份以获得专业化回答

3.2 提示类型分类

Table 1: 提示工程技术对比

技术	适用场景	优点	缺点
Zero-shot	简单任务	简洁高效	效果有限
Few-shot	需要示例引导	效果较好	消耗更多 token
Chain-of-Thought	推理任务	逻辑清晰	响应较长
Role-based	专业领域	专业性强	可能过度拟人化

3.3 实践示例

3.3.1 零样本提示 (Zero-shot)

```
1 # 最简单的提示形式，不提供任何示例
2 def zero_shot_prompt(task_description):
3     return f"请完成以下任务：{task_description}"
4
5 # 示例使用
6 prompt = zero_shot_prompt("将以下句子翻译成英文：你好世界")
7 print(prompt)
8 # 输出：请完成以下任务：将以下句子翻译成英文：你好世界
```

Listing 1: 零样本提示示例

3.3.2 少样本提示 (Few-shot)

```
1 def few_shot_prompt(examples, new_input):
2     """
3     构建包含示例的提示
4     Args:
5         examples: 示例列表，每个示例包含输入和输出
6         new_input: 新的输入，需要模型处理
7     """
8     prompt = "以下是一些示例：\n\n"
9
10    # 添加示例
11    for i, example in enumerate(examples, 1):
12        prompt += f"示例{i}:\n"
13        prompt += f"输入：{example['input']}\n"
14        prompt += f"输出：{example['output']}\n\n"
15
```

```
16 # 添加新任务
17 prompt += f"现在处理新任务:\n输入: {new_input}\n输出: "
18
19 return prompt
20
21 # 示例使用: 情感分析任务
22 examples = [
23     {"input": "这部电影太精彩了!", "output": "积极"},
24     {"input": "服务态度很差, 不推荐", "output": "消极"},
25     {"input": "价格还可以, 性价比一般", "output": "中性"}
26 ]
27
28 prompt = few_shot_prompt(examples, "今天天气不错, 心情很好")
29 print(prompt)
```

Listing 2: 少样本提示示例

3.3.3 思维链提示 (Chain-of-Thought)

```
1 def chain_of_thought_prompt(problem):
2     """
3     构建思维链提示, 引导模型逐步推理
4     """
5     template = """
6 请按以下步骤分析问题:
7
8 问题: {problem}
9
10 分析步骤:
11 1. 理解问题: 明确要解决的核心问题是什么
12 2. 识别关键信息: 从问题中提取重要的数据和条件
13 3. 制定解决方案: 选择合适的方法或公式
14 4. 逐步计算: 展示详细的计算过程
15 5. 验证结果: 检查答案是否合理
16 6. 给出结论: 简洁地回答原始问题
17
18 让我们开始分析:
19 """
20     return template.format(problem=problem)
21
22 # 示例使用: 数学问题
23 math_problem = "一个长方形的长是12米, 宽是8米, 求其面积和周长"
24 prompt = chain_of_thought_prompt(math_problem)
25 print(prompt)
```

Listing 3: 思维链提示示例

3.4 上下文管理技术

3.4.1 Token 计算与管理

```
1 import tiktoken
2 from typing import List, Dict
3
4 class ContextManager:
```

```

5     def __init__(self, model_name: str = "gpt-3.5-turbo", max_tokens:
6         int = 4000):
7         """
8         初始化上下文管理器
9         Args:
10            model_name: 模型名称, 用于选择正确的tokenizer
11            max_tokens: 最大token限制
12            """
13            self.encoding = tiktoken.encoding_for_model(model_name)
14            self.max_tokens = max_tokens
15            self.reserved_tokens = 500 # 为响应预留的token数
16
17        def count_tokens(self, text: str) -> int:
18            """计算文本的token数量"""
19            return len(self.encoding.encode(text))
20
21        def optimize_context(self, messages: List[Dict[str, str]]) -> List[
22            Dict[str, str]]:
23            """
24            优化上下文, 确保不超过token限制
25            Args:
26                messages: 消息列表, 格式为 [{"role": "user", "content": "
27                ..."}]
28            """
29            total_tokens = sum(self.count_tokens(msg["content"]) for msg in
30                messages)
31
32            if total_tokens <= self.max_tokens - self.reserved_tokens:
33                return messages # 不需要优化
34
35            # 优化策略: 保留系统消息和最近的对话
36            optimized_messages = []
37            current_tokens = 0
38
39            # 首先保留系统消息
40            for msg in messages:
41                if msg["role"] == "system":
42                    optimized_messages.append(msg)
43                    current_tokens += self.count_tokens(msg["content"])
44
45            # 从最新消息开始向前保留
46            for msg in reversed(messages):
47                if msg["role"] != "system":
48                    msg_tokens = self.count_tokens(msg["content"])
49                    if current_tokens + msg_tokens <= self.max_tokens -
50                        self.reserved_tokens:
51                        optimized_messages.insert(-len([m for m in
52                            optimized_messages if m["role"] == "system"]), msg)
53                        current_tokens += msg_tokens
54                    else:
55                        break
56
57            return optimized_messages
58
59        def sliding_window_context(self, messages: List[Dict[str, str]],
60            window_size: int = 10) -> List[Dict[str, str]]:
61            """

```

```
55     滑动窗口上下文管理
56     保留最近的N条消息
57     """
58     if len(messages) <= window_size:
59         return messages
60
61     # 保留系统消息和最近的N条消息
62     system_messages = [msg for msg in messages if msg["role"] == "
system"]
63     recent_messages = messages[-window_size:]
64
65     # 去重系统消息
66     for msg in recent_messages:
67         if msg["role"] == "system":
68             if msg not in system_messages:
69                 system_messages.append(msg)
70
71     # 构建最终消息列表
72     final_messages = system_messages + [msg for msg in
recent_messages if msg["role"] != "system"]
73
74     return final_messages
75
76 # 使用示例
77 context_manager = ContextManager()
78
79 messages = [
80     {"role": "system", "content": "你是一个有用的AI助手"},
81     {"role": "user", "content": "什么是机器学习?"},
82     {"role": "assistant", "content": "机器学习是人工智能的一个分支..."
},
83     {"role": "user", "content": "能举个例子吗?"},
84     {"role": "assistant", "content": "当然可以, 比如推荐系统..."}
85 ]
86
87 # 计算总token数
88 total_tokens = sum(context_manager.count_tokens(msg["content"]) for msg
in messages)
89 print(f"总token数: {total_tokens}")
90
91 # 优化上下文
92 optimized = context_manager.optimize_context(messages)
93 print(f"优化后消息数: {len(optimized)}")
```

Listing 4: Token 计算和上下文管理

4 Model Context Protocol (MCP)

MCP 是一种标准化协议, 用于 LLM 应用程序和外部数据源及工具之间的连接。

4.1 MCP 架构原理

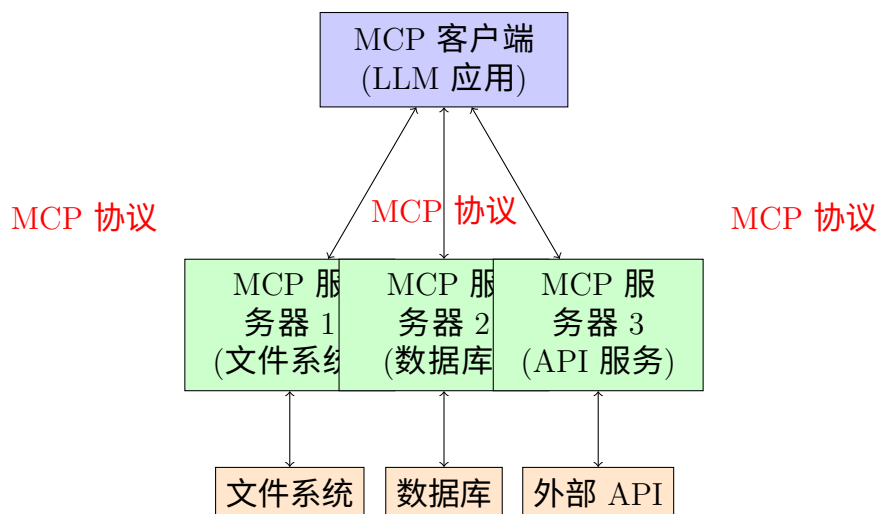


Figure 2: MCP 架构图

4.2 MCP 核心组件

1. Resources: 服务器可以暴露的数据资源
2. Tools: 客户端可以调用的功能工具
3. Prompts: 预定义的提示模板

4.3 MCP 服务器实现

```
1 from typing import Dict, List, Any, Optional
2 import asyncio
3 import json
4 from dataclasses import dataclass
5
6 @dataclass
7 class Resource:
8     """MCP资源定义"""
9     uri: str
10    name: str
11    description: str
12    mime_type: str = "text/plain"
13
14 @dataclass
15 class Tool:
16     """MCP工具定义"""
17     name: str
18     description: str
19     input_schema: Dict[str, Any]
20
21 @dataclass
22 class ToolResult:
23     """工具执行结果"""
24     content: List[Dict[str, Any]]
```

```
25     is_error: bool = False
26
27 class MCPServer:
28     """MCP服务器基础类"""
29
30     def __init__(self, name: str, version: str = "1.0.0"):
31         self.name = name
32         self.version = version
33         self.resources: Dict[str, Resource] = {}
34         self.tools: Dict[str, Tool] = {}
35
36     def register_resource(self, resource: Resource):
37         """注册资源"""
38         self.resources[resource.uri] = resource
39
40     def register_tool(self, tool: Tool, handler):
41         """注册工具和其处理函数"""
42         self.tools[tool.name] = tool
43         setattr(self, f"_handle_{tool.name}", handler)
44
45     async def list_resources(self) -> List[Resource]:
46         """列出所有可用资源"""
47         return list(self.resources.values())
48
49     async def read_resource(self, uri: str) -> Dict[str, Any]:
50         """读取指定资源"""
51         if uri not in self.resources:
52             raise ValueError(f"Resource not found: {uri}")
53
54         resource = self.resources[uri]
55         # 这里应该实现具体的资源读取逻辑
56         content = await self._read_resource_content(uri)
57
58         return {
59             "contents": [{
60                 "uri": uri,
61                 "mimeType": resource.mime_type,
62                 "text": content
63             }]
64         }
65
66     async def list_tools(self) -> List[Tool]:
67         """列出所有可用工具"""
68         return list(self.tools.values())
69
70     async def call_tool(self, name: str, arguments: Dict[str, Any]) -> ToolResult:
71         """调用指定工具"""
72         if name not in self.tools:
73             return ToolResult(
74                 content=[{"type": "text", "text": f"Tool not found: {
name}" }],
75                 is_error=True
76             )
77
78         try:
79             handler = getattr(self, f"_handle_{name}")
```

```

80         result = await handler(arguments)
81         return ToolResult(content=result)
82     except Exception as e:
83         return ToolResult(
84             content=[{"type": "text", "text": f"Tool execution
failed: {str(e)}"}],
85             is_error=True
86         )
87
88     async def _read_resource_content(self, uri: str) -> str:
89         """子类应重写此方法来实现具体的资源读取逻辑"""
90         raise NotImplementedError
91
92 # 文件系统MCP服务器示例
93 class FileSystemMCPServer(MCPServer):
94     """文件系统MCP服务器"""
95
96     def __init__(self, base_path: str):
97         super().__init__("filesystem-server")
98         self.base_path = base_path
99         self._setup_resources_and_tools()
100
101     def _setup_resources_and_tools(self):
102         """设置资源和工具"""
103         import os
104
105         # 注册文件资源
106         for root, dirs, files in os.walk(self.base_path):
107             for file in files:
108                 if file.endswith(('.txt', '.md', '.py', '.json')):
109                     file_path = os.path.join(root, file)
110                     relative_path = os.path.relpath(file_path, self.
base_path)
111
112                     resource = Resource(
113                         uri=f"file://{relative_path}",
114                         name=file,
115                         description=f"File: {relative_path}",
116                         mime_type=self._get_mime_type(file)
117                     )
118                     self.register_resource(resource)
119
120         # 注册搜索工具
121         search_tool = Tool(
122             name="search_files",
123             description="在文件中搜索指定内容",
124             input_schema={
125                 "type": "object",
126                 "properties": {
127                     "query": {
128                         "type": "string",
129                         "description": "搜索关键词"
130                     },
131                     "file_pattern": {
132                         "type": "string",
133                         "description": "文件模式匹配, 如*.py",
134                         "default": "*"

```

```

135         }
136     },
137     "required": ["query"]
138 }
139 )
140 self.register_tool(search_tool, self._search_files)
141
142 def _get_mime_type(self, filename: str) -> str:
143     """根据文件扩展名确定MIME类型"""
144     ext = filename.lower().split('.')[-1]
145     mime_types = {
146         'txt': 'text/plain',
147         'md': 'text/markdown',
148         'py': 'text/x-python',
149         'json': 'application/json',
150         'js': 'text/javascript',
151         'html': 'text/html'
152     }
153     return mime_types.get(ext, 'text/plain')
154
155 async def _read_resource_content(self, uri: str) -> str:
156     """读取文件内容"""
157     import os
158     file_path = uri.replace("file://", "")
159     full_path = os.path.join(self.base_path, file_path)
160
161     try:
162         with open(full_path, 'r', encoding='utf-8') as f:
163             return f.read()
164     except Exception as e:
165         raise ValueError(f"Cannot read file {file_path}: {str(e)}")
166
167 async def _search_files(self, arguments: Dict[str, Any]) -> List[
    Dict[str, Any]]:
168     """搜索文件内容"""
169     import os
170     import fnmatch
171
172     query = arguments["query"].lower()
173     pattern = arguments.get("file_pattern", "*")
174     results = []
175
176     for root, dirs, files in os.walk(self.base_path):
177         for file in files:
178             if fnmatch.fnmatch(file, pattern):
179                 file_path = os.path.join(root, file)
180                 try:
181                     with open(file_path, 'r', encoding='utf-8') as
182 f:
183                     content = f.read()
184                     if query in content.lower():
185                         relative_path = os.path.relpath(
186 file_path, self.base_path)
187                         results.append({
188                             "type": "text",
189                             "text": f"Found in {relative_path}:\n{self._extract_context(content, query)}"
190                         })

```

```

189         except:
190             continue # 跳过无法读取的文件
191
192     if not results:
193         results.append({
194             "type": "text",
195             "text": f"No results found for query: {query}"
196         })
197
198     return results
199
200     def _extract_context(self, content: str, query: str, context_size:
201     int = 100) -> str:
202         """提取搜索结果的上下文"""
203         lines = content.split('\n')
204         results = []
205
206         for i, line in enumerate(lines):
207             if query.lower() in line.lower():
208                 start = max(0, i - 2)
209                 end = min(len(lines), i + 3)
210                 context_lines = lines[start:end]
211                 results.append(f"Line {i+1}:\n" + '\n'.join(
212                     context_lines))
213
214         return '\n\n'.join(results[:3]) # 最多返回3个匹配结果
215
216 # 使用示例
217 async def main():
218     # 创建文件系统MCP服务器
219     server = FileSystemMCPServer("/path/to/documents")
220
221     # 列出资源
222     resources = await server.list_resources()
223     print(f"Available resources: {len(resources)}")
224     for resource in resources[:5]: # 显示前5个
225         print(f"    - {resource.name}: {resource.uri}")
226
227     # 列出工具
228     tools = await server.list_tools()
229     print(f"\nAvailable tools: {[tool.name for tool in tools]}")
230
231     # 使用搜索工具
232     search_result = await server.call_tool("search_files", {
233         "query": "function",
234         "file_pattern": "*.py"
235     })
236
237     print(f"\nSearch results:")
238     for content in search_result.content:
239         print(content["text"])
240
241 # 运行示例
242 # asyncio.run(main())

```

Listing 5: MCP 服务器基础实现

5 多智能体系统

多智能体系统允许多个 AI 智能体协作完成复杂任务，每个智能体可以有不同的专业能力和角色。

5.1 核心概念

- **智能体 (Agent)**: 具有特定角色和能力的 AI 实体
- **协作模式**: 智能体之间的交互方式
- **任务分配**: 如何将复杂任务分解给不同智能体
- **结果聚合**: 如何整合多个智能体的输出

5.2 AutoGen 框架

AutoGen 是微软开源的多智能体对话框架。

5.2.1 基本架构

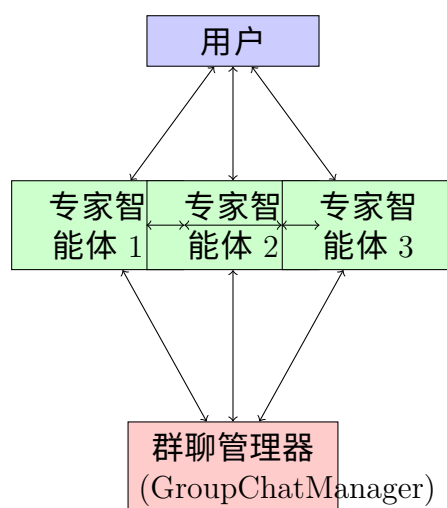


Figure 3: AutoGen 多智能体架构

5.2.2 AutoGen 实现示例

```
1 import autogen
2 from typing import List, Dict, Any
3
4 # 配置 LLM 连接
5 config_list = [
6     {
7         "model": "gpt-4",
8         "api_key": "your-api-key",
9         "api_base": "https://api.openai.com/v1",
10        "api_type": "openai"
11    }
12 ]
```

```
13
14 llm_config = {
15     "config_list": config_list,
16     "temperature": 0.7,
17     "timeout": 60,
18 }
19
20 class BuildingProjectAnalysisSystem:
21     """建筑项目分析多智能体系统"""
22
23     def __init__(self):
24         self.setup_agents()
25         self.setup_group_chat()
26
27     def setup_agents(self):
28         """设置专业智能体"""
29
30         # 建筑师智能体 - 专注设计和美学
31         self.architect = autogen.ConversableAgent(
32             name="architect",
33             system_message="""你是一位有20年经验的资深建筑师。你的专长
包括：
- 建筑设计原理和美学评估
- 空间规划和功能性分析
- 建筑规范和法规符合性检查
- 可持续性和绿色建筑评估
请用专业但易懂的语言提供建议，并总是考虑项目的整体设计理念。
""",
34             llm_config=llm_config,
35             human_input_mode="NEVER",
36             max_consecutive_auto_reply=3,
37         )
38
39         # 结构工程师智能体 - 专注技术可行性
40         self.structural_engineer = autogen.ConversableAgent(
41             name="structural_engineer",
42             system_message="""你是一位专业的结构工程师，具有15年工程经验。你的职责包括：
- 结构安全性和稳定性评估
- 材料选择和荷载计算
- 施工可行性分析
- 抗震和抗风设计评估
请提供技术准确的分析，包含具体的工程数据和计算依据。
""",
43             llm_config=llm_config,
44             human_input_mode="NEVER",
45             max_consecutive_auto_reply=3,
46         )
47
48         # 项目经理智能体 - 专注成本和进度
49         self.project_manager = autogen.ConversableAgent(
50             name="project_manager",
51             system_message="""你是经验丰富的建筑项目经理，擅长：
52
53
54
55
56
57
58
59
60
61
62
63
64
65
```

```

66         - 项目成本估算和预算控制
67         - 进度规划和里程碑管理
68         - 资源分配和团队协作
69         - 风险识别和缓解策略
70
71         请从项目管理角度提供实用的建议，关注可执行性和商业价值。
72         """
73         llm_config=llm_config,
74         human_input_mode="NEVER",
75         max_consecutive_auto_reply=3,
76     )
77
78     # 质量控制专家智能体
79     self.quality_expert = autogen.ConversableAgent(
80         name="quality_expert",
81         system_message="""你是建筑质量控制专家，负责：
82         - 施工质量标准制定
83         - 材料质量检测要求
84         - 工艺流程质量把控
85         - 验收标准和检查清单
86
87         请提供详细的质量保证措施和检测方案。
88         """,
89         llm_config=llm_config,
90         human_input_mode="NEVER",
91         max_consecutive_auto_reply=2,
92     )
93
94     # 用户代理 - 代表客户提出问题
95     self.user_proxy = autogen.ConversableAgent(
96         name="user_proxy",
97         system_message="你代表项目客户，负责提出问题并汇总专家意见。",
98         llm_config=llm_config,
99         human_input_mode="ALWAYS", # 允许人工输入
100         max_consecutive_auto_reply=0,
101         code_execution_config={"work_dir": "project_analysis"}
102     )
103
104     def setup_group_chat(self):
105         """设置群聊管理"""
106
107         # 定义发言者选择逻辑
108         def custom_speaker_selection(last_speaker, groupchat):
109             """自定义发言者选择逻辑"""
110             messages = groupchat.messages
111
112             if len(messages) == 1: # 第一条消息后
113                 return self.architect # 建筑师先发言
114
115             last_message = messages[-1]["content"].lower()
116
117             # 根据消息内容选择下一个发言者
118             if "成本" in last_message or "预算" in last_message:
119                 return self.project_manager
120             elif "结构" in last_message or "安全" in last_message:

```



```

121         return self.structural_engineer
122     elif "质量" in last_message or "检测" in last_message:
123         return self.quality_expert
124     elif "设计" in last_message or "美观" in last_message:
125         return self.architect
126     else:
127         # 轮询选择
128         agents = [self.architect, self.structural_engineer,
129                   self.project_manager, self.quality_expert]
130         current_speaker_idx = agents.index(last_speaker) if
last_speaker in agents else 0
131         next_idx = (current_speaker_idx + 1) % len(agents)
132         return agents[next_idx]
133
134     # 创建群聊
135     self.group_chat = autogen.GroupChat(
136         agents=[
137             self.user_proxy,
138             self.architect,
139             self.structural_engineer,
140             self.project_manager,
141             self.quality_expert
142         ],
143         messages=[],
144         max_round=15, # 最大对话轮数
145         speaker_selection_method=custom_speaker_selection,
146         allow_repeat_speaker=False # 避免同一智能体连续发言
147     )
148
149     # 创建群聊管理器
150     self.group_chat_manager = autogen.GroupChatManager(
151         groupchat=self.group_chat,
152         llm_config=llm_config,
153         system_message="""你是多专家协作系统的管理者。你的职责是：
154         1. 确保每个专家都能充分表达专业观点
155         2. 引导讨论朝向问题解决的方向
156         3. 在适当时机总结讨论成果
157         4. 识别专家间观点冲突并促进解决
158
159         请保持中立，促进建设性对话。
160         """
161     )
162
163     def analyze_project(self, project_description: str) -> str:
164         """分析建筑项目"""
165
166         analysis_prompt = f"""
167         请对以下建筑项目进行全面分析：
168
169         项目描述：{project_description}
170
171         请各位专家从自己的专业角度分析：
172         1. 项目的可行性和潜在风险
173         2. 关键技术要求和质量标准
174         3. 成本估算和时间规划
175         4. 优化建议和改进方案

```

```

176
177     最后需要形成统一的项目评估报告。
178     """
179
180     # 启动群聊分析
181     result = self.user_proxy.initiate_chat(
182         self.group_chat_manager,
183         message=analysis_prompt,
184         clear_history=True
185     )
186
187     return result
188
189 # 使用示例
190 def demonstrate_multi_agent_analysis():
191     """演示多智能体分析系统"""
192
193     # 创建分析系统
194     analysis_system = BuildingProjectAnalysisSystem()
195
196     # 示例项目
197     project_description = """
198     项目名称：绿色办公综合体
199     项目规模：地上25层，地下3层，总建筑面积8万平方米
200     功能定位：甲级写字楼 + 商业裙楼 + 地下停车场
201     特殊要求：LEED金级认证，智能化程度要求高
202     预算范围：15-20亿人民币
203     建设周期：36个月
204     地理位置：一线城市CBD核心区域
205     """
206
207     print("=== 多智能体建筑项目分析系统 ===\n")
208     print(f"项目描述：{project_description}\n")
209     print("正在启动专家团队分析...\n")
210
211     # 执行分析
212     result = analysis_system.analyze_project(project_description)
213
214     print("=== 分析完成 ===")
215     return result
216
217 # 运行演示
218 # result = demonstrate_multi_agent_analysis()

```

Listing 6: AutoGen 多智能体协作系统

5.3 CrewAI 任务协作模式

CrewAI 专注于任务驱动的智能体协作。

```

1 from crewai import Agent, Task, Crew, Process
2 from crewai.tools import tool
3 from typing import List
4 import os
5
6 # 定义自定义工具

```

```
7 @tool("search_building_codes")
8 def search_building_codes(query: str) -> str:
9     """搜索建筑规范和标准"""
10    # 这里应该连接真实的建筑规范数据库
11    mock_results = {
12        "防火": "根据《建筑设计防火规范》GB50016-2014，高层建筑应设置自
13        动喷水灭火系统...",
14        "结构": "按照《建筑结构荷载规范》GB50009-2012，办公建筑楼面活荷
15        载标准值为2.0kN/m²...",
16        "节能": "依据《公共建筑节能设计标准》GB50189-2015，建筑围护结构
17        热工性能应符合..."
18    }
19
20    for keyword, result in mock_results.items():
21        if keyword in query:
22            return result
23
24    return f"未找到关于'{query}'的相关规范信息"
25
26 @tool("calculate_construction_cost")
27 def calculate_construction_cost(area: float, building_type: str,
28    location: str) -> str:
29    """计算建设成本估算"""
30    # 简化的成本计算逻辑
31    base_costs = {
32        "office": 4000, # 元/平方米
33        "commercial": 5000,
34        "residential": 3000
35    }
36
37    location_multipliers = {
38        "一线城市": 1.3,
39        "二线城市": 1.1,
40        "三线城市": 1.0
41    }
42
43    base_cost = base_costs.get(building_type.lower(), 4000)
44    multiplier = location_multipliers.get(location, 1.0)
45    total_cost = area * base_cost * multiplier
46
47    return f"建设成本估算: {total_cost:,.0f}元 (按{base_cost}元/㎡计
48    算, 地区调整系数{multiplier})"
49
50 class BuildingProjectCrew:
51     """建筑项目CrewAI团队"""
52
53     def __init__(self):
54         self.setup_agents()
55         self.setup_tasks()
56
57     def setup_agents(self):
58         """设置智能体团队"""
59
60         # 研究分析师 - 收集项目相关信息
61         self.research_analyst = Agent(
62             role='建筑项目研究分析师',
```

```
58         goal='收集和分析项目的技术要求、法规要求和市场信息',
59         backstory="""你是一位经验丰富的建筑行业研究分析师，擅长：
60         - 建筑法规和标准研究
61         - 市场趋势和技术发展分析
62         - 项目可行性研究
63         - 风险评估和机会识别
64         你总是能找到项目需要的关键信息和数据。""",
65         tools=[search_building_codes],
66         verbose=True,
67         allow_delegation=False
68     )
69
70     # 设计工程师 - 负责技术方案设计
71     self.design_engineer = Agent(
72         role='建筑设计工程师',
73         goal='基于项目需求设计最优的技术解决方案',
74         backstory="""你是资深的建筑设计工程师，具有以下专长：
75         - 建筑方案设计和优化
76         - 结构系统选择和设计
77         - 机电系统集成设计
78         - 绿色建筑和智能化设计
79         你能将复杂的技术要求转化为可行的设计方案。""",
80         verbose=True,
81         allow_delegation=True # 可以委托给其他智能体
82     )
83
84     # 成本工程师 - 负责成本分析和控制
85     self.cost_engineer = Agent(
86         role='建设成本工程师',
87         goal='提供准确的成本估算和成本控制方案',
88         backstory="""你是专业的建设成本工程师，精通：
89         - 工程量计算和成本估算
90         - 材料和人工价格分析
91         - 成本控制和优化策略
92         - 合同和采购管理
93         你的估算准确度高，能有效控制项目成本。""",
94         tools=[calculate_construction_cost],
95         verbose=True,
96         allow_delegation=False
97     )
98
99     # 项目协调员 - 统筹协调和报告编写
100    self.project_coordinator = Agent(
101        role='项目协调员',
102        goal='协调各专业团队，编写综合项目报告',
103        backstory="""你是经验丰富的项目协调员，负责：
104        - 跨专业团队协作
105        - 项目信息整合
106        - 综合报告编写
107        - 风险和问题管理
108        你擅长将不同专业的观点整合成完整的项目方案。""",
109        verbose=True,
110        allow_delegation=True
111    )
112
```

```
113     def setup_tasks(self):
114         """设置协作任务"""
115         pass # 任务将在分析方法中动态创建
116
117     def analyze_building_project(self, project_info: Dict[str, Any]) ->
118         str:
119         """分析建筑项目"""
120
121         # 任务1: 项目研究和法规分析
122         research_task = Task(
123             description=f"""
124             对以下建筑项目进行深入研究:
125
126             项目信息: {project_info}
127
128             请完成:
129             1. 分析适用的建筑法规和标准
130             2. 研究类似项目的成功案例
131             3. 识别项目的技术难点和风险
132             4. 提出初步的解决思路
133
134             输出详细的研究报告。
135             """,
136             expected_output='包含法规分析、案例研究和风险评估的详细研究
137             报告',
138             agent=self.research_analyst,
139             tools=[search_building_codes]
140         )
141
142         # 任务2: 技术方案设计
143         design_task = Task(
144             description="""
145             基于研究分析的结果, 设计项目的技术解决方案:
146
147             1. 建筑设计方案 (布局、立面、空间组织)
148             2. 结构系统选择和设计要点
149             3. 机电系统配置方案
150             4. 绿色建筑和智能化策略
151             5. 施工技术和工艺要求
152
153             提供具体可行的技术方案。
154             """,
155             expected_output='完整的技术设计方案文档, 包含各专业设计要点
156             ',
157             agent=self.design_engineer,
158             context=[research_task] # 依赖研究任务的结果
159         )
160
161         # 任务3: 成本分析
162         cost_task = Task(
163             description=f"""
164             根据技术方案进行详细的成本分析:
165
166             项目规模: {project_info.get('area', 0)}平方米
167             建筑类型: {project_info.get('type', '办公建筑')}
168             """
169         )
```

```

165         项目位置: {project_info.get('location', '一线城市')}
166
167         分析内容:
168         1. 各专业工程成本估算
169         2. 材料和设备成本分析
170         3. 人工和管理费用
171         4. 成本优化建议
172         5. 成本控制关键点
173         """
174         expected_output='详细的成本分析报告，包含分项成本和优化建议
175     ',
176         agent=self.cost_engineer,
177         context=[design_task], # 依赖设计任务
178         tools=[calculate_construction_cost]
179     )
180
181     # 任务4: 综合项目报告
182     final_report_task = Task(
183         description="""
184         整合所有专业分析结果，编写综合项目评估报告：
185
186         1. 项目概况和目标
187         2. 技术方案总结
188         3. 成本估算和经济分析
189         4. 风险评估和应对措施
190         5. 项目实施建议
191         6. 结论和推荐方案
192
193         报告应该结构清晰，便于决策者理解。
194         """,
195         expected_output='结构完整的项目评估报告，包含明确的结论和建
196         议',
197         agent=self.project_coordinator,
198         context=[research_task, design_task, cost_task] # 依赖所有
199         前置任务
200     )
201
202     # 创建团队并执行任务
203     crew = Crew(
204         agents=[
205             self.research_analyst,
206             self.design_engineer,
207             self.cost_engineer,
208             self.project_coordinator
209         ],
210         tasks=[
211             research_task,
212             design_task,
213             cost_task,
214             final_report_task
215         ],
216         process=Process.sequential, # 顺序执行任务
217         verbose=2 # 详细输出
218     )
219
220     # 执行协作任务

```

```
218         result = crew.kickoff()
219         return result
220
221 # 使用示例
222 def demonstrate_crewai_collaboration():
223     """演示CrewAI协作系统"""
224
225     # 创建项目团队
226     project_crew = BuildingProjectCrew()
227
228     # 示例项目信息
229     project_info = {
230         "name": "智能绿色办公园区",
231         "area": 120000, # 平方米
232         "floors": 28,
233         "type": "office",
234         "location": "一线城市",
235         "budget": "25亿元",
236         "timeline": "42个月",
237         "special_requirements": [
238             "LEED铂金级认证",
239             "智能化5A标准",
240             "装配式建筑技术",
241             "地标性建筑设计"
242         ]
243     }
244
245     print("=== CrewAI建筑项目协作分析 ===\n")
246     print(f"项目: {project_info['name']}")
247     print(f"规模: {project_info['area']:,}平方米")
248     print(f"预算: {project_info['budget']}")
249     print(f"工期: {project_info['timeline']}\n")
250
251     print("正在启动专业团队协作分析...\n")
252
253     # 执行协作分析
254     result = project_crew.analyze_building_project(project_info)
255
256     print("=== 协作分析完成 ===")
257     print(f"最终报告: \n{result}")
258
259     return result
260
261 # 运行演示
262 # result = demonstrate_crewai_collaboration()
```

Listing 7: CrewAI 任务协作系统

6 向量数据库技术基础

向量数据库是专门为存储和检索高维向量而设计的数据库系统，是现代 AI 应用的核心基础设施。

6.1 向量的数学基础

6.1.1 向量表示

在 AI 应用中，文本、图像等数据通过嵌入 (Embedding) 技术转换为高维向量：

$$\mathbf{v} = [v_1, v_2, \dots, v_d] \in \mathbb{R}^d \quad (6)$$

其中 d 是向量维度 (通常为 128、256、512、768、1536 等)。

6.1.2 相似度计算

向量间相似度的常用计算方法：

余弦相似度

$$\text{cosine}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}| \times |\mathbf{b}|} = \frac{\sum_{i=1}^d a_i b_i}{\sqrt{\sum_{i=1}^d a_i^2} \times \sqrt{\sum_{i=1}^d b_i^2}} \quad (7)$$

欧氏距离

$$\text{euclidean}(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{i=1}^d (a_i - b_i)^2} \quad (8)$$

内积 (点积)

$$\text{dot_product}(\mathbf{a}, \mathbf{b}) = \sum_{i=1}^d a_i b_i = \mathbf{a} \cdot \mathbf{b} \quad (9)$$

6.2 向量索引算法

6.2.1 HNSW 算法原理

HNSW(Hierarchical Navigable Small World) 是目前最流行的近似最近邻搜索算法。

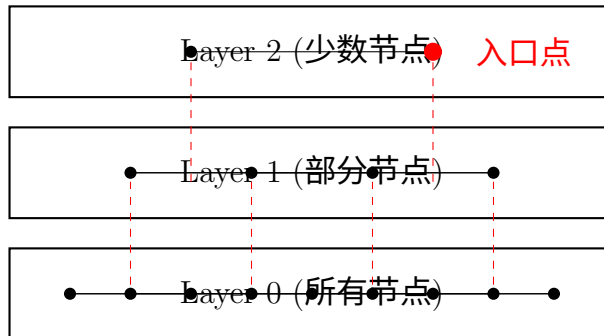


Figure 4: HNSW 多层索引结构

HNSW 算法的核心思想：

1. 构建多层图结构，上层节点稀疏，下层节点稠密
2. 从最高层的入口点开始搜索

3. 在每层找到最近的邻居后，下降到下一层继续搜索
4. 在最底层进行精确搜索

6.2.2 HNSW 参数优化

Table 2: HNSW 关键参数说明

参数	含义	影响	推荐值
M	每个节点的连接数	影响召回率和内存	16-32
efConstruction	构建时搜索宽度	影响构建质量	100-400
efSearch	搜索时探索宽度	影响搜索精度	50-200
maxM	最大连接数	限制内存使用	通常等于 M
maxM0	第 0 层最大连接数	底层连接密度	2CEM

6.3 Qdrant 实战应用

Qdrant 是一个高性能的向量数据库，专为 AI 应用设计。

```

1 from qdrant_client import QdrantClient
2 from qdrant_client.http import models
3 from qdrant_client.http.models import (
4     Distance, VectorParams, OptimizersConfig,
5     HnswConfig, QuantizationConfig, ScalarQuantization
6 )
7 import numpy as np
8 from sentence_transformers import SentenceTransformer
9 from typing import List, Dict, Any
10 import time
11
12 class AdvancedQdrantManager:
13     """高级Qdrant管理器"""
14
15     def __init__(self, host: str = "localhost", port: int = 6333):
16         """初始化Qdrant客户端"""
17         self.client = QdrantClient(host=host, port=port)
18         self.encoder = SentenceTransformer('all-MiniLM-L6-v2')
19         self.vector_size = 384 # all-MiniLM-L6-v2的向量维度
20
21     def create_optimized_collection(
22         self,
23         collection_name: str,
24         vector_size: int = None,
25         distance: Distance = Distance.COSINE,
26         storage_type: str = "memory" # "memory" 或 "disk"
27     ):
28         """创建优化的集合配置"""
29
30         if vector_size is None:
31             vector_size = self.vector_size
32
33         # 根据数据规模选择优化配置
34         if storage_type == "disk":
35             # 大数据集，优化磁盘存储

```

```

36         optimizers_config = OptimizersConfig(
37             default_segment_number=4, # 增加段数提高并发
38             max_segment_size=100000, # 较小的段大小
39             memmap_threshold=50000, # 内存映射阈值
40             indexing_threshold=20000, # 索引阈值
41             flush_interval_sec=10, # 刷新间隔
42             max_optimization_threads=2
43         )
44
45         hnsw_config = HnswConfig(
46             m=16, # 适中的连接数
47             ef_construct=200, # 较高的构建精度
48             full_scan_threshold=10000,
49             max_indexing_threads=2,
50             on_disk=True, # 索引存储在磁盘
51             payload_m=None
52         )
53     else:
54         # 小数据集，优化内存性能
55         optimizers_config = OptimizersConfig(
56             default_segment_number=2,
57             max_segment_size=200000,
58             memmap_threshold=1000000,
59             indexing_threshold=10000,
60             flush_interval_sec=5,
61             max_optimization_threads=4
62         )
63
64         hnsw_config = HnswConfig(
65             m=32, # 更高的连接数
66             ef_construct=100, # 平衡构建时间和质量
67             full_scan_threshold=5000,
68             max_indexing_threads=0, # 使用所有可用线程
69             on_disk=False, # 索引在内存中
70             payload_m=16
71         )
72
73         # 量化配置以减少内存使用
74         quantization_config = QuantizationConfig(
75             scalar=ScalarQuantization(
76                 type=models.ScalarType.INT8, # 8位量化
77                 quantile=0.99, # 量化分位数
78                 always_ram=True # 量化数据常驻内存
79             )
80         )
81
82         # 创建集合
83         try:
84             self.client.create_collection(
85                 collection_name=collection_name,
86                 vectors_config=VectorParams(
87                     size=vector_size,
88                     distance=distance,
89                     on_disk=storage_type == "disk"
90                 ),
91                 optimizers_config=optimizers_config,

```

```
92         hnsw_config=hnsw_config,
93         quantization_config=quantization_config,
94         shard_number=1,           # 单分片开始
95         replication_factor=1      # 无副本
96     )
97     print(f"集合 '{collection_name}' 创建成功")
98
99     except Exception as e:
100         print(f"创建集合失败: {e}")
101
102     def batch_upsert_documents(
103         self,
104         collection_name: str,
105         documents: List[str],
106         metadatas: List[Dict] = None,
107         batch_size: int = 100
108     ):
109         """批量插入文档"""
110
111         if metadatas is None:
112             metadatas = [{}] * len(documents)
113
114         points = []
115
116         print(f"开始编码 {len(documents)} 个文档...")
117         start_time = time.time()
118
119         # 批量编码文档
120         for i in range(0, len(documents), batch_size):
121             batch_docs = documents[i:i + batch_size]
122             batch_meta = metadatas[i:i + batch_size]
123
124             # 编码当前批次
125             batch_vectors = self.encoder.encode(batch_docs).tolist()
126
127             # 创建点对象
128             for j, (doc, vector, meta) in enumerate(zip(batch_docs,
129 batch_vectors, batch_meta)):
130                 point_id = i + j
131
132                 # 合并文档内容到元数据
133                 payload = {
134                     **meta,
135                     "text": doc,
136                     "length": len(doc),
137                     "created_at": time.time()
138                 }
139
140                 points.append(models.PointStruct(
141                     id=point_id,
142                     vector=vector,
143                     payload=payload
144                 ))
145
146         encoding_time = time.time() - start_time
147         print(f"编码完成, 耗时: {encoding_time:.2f}秒")
```

```
148     # 批量上传到Qdrant
149     print("开始上传到Qdrant...")
150     upload_start = time.time()
151
152     try:
153         self.client.upsert(
154             collection_name=collection_name,
155             points=points,
156             wait=True  # 等待操作完成
157         )
158
159         upload_time = time.time() - upload_start
160         print(f"上传完成, 耗时: {upload_time:.2f}秒")
161         print(f"总计处理 {len(documents)} 个文档")
162
163     except Exception as e:
164         print(f"上传失败: {e}")
165
166     def hybrid_search(
167         self,
168         collection_name: str,
169         query: str,
170         limit: int = 10,
171         score_threshold: float = 0.7,
172         filter_conditions: Dict = None
173     ) -> List[Dict]:
174         """混合搜索: 向量搜索 + 过滤"""
175
176         # 编码查询
177         query_vector = self.encoder.encode([query])[0].tolist()
178
179         # 构建过滤条件
180         search_filter = None
181         if filter_conditions:
182             filter_clauses = []
183
184             for key, value in filter_conditions.items():
185                 if isinstance(value, str):
186                     # 文本匹配
187                     filter_clauses.append(
188                         models.FieldCondition(
189                             key=key,
190                             match=models.MatchValue(value=value)
191                         )
192                     )
193                 elif isinstance(value, (int, float)):
194                     # 数值匹配
195                     filter_clauses.append(
196                         models.FieldCondition(
197                             key=key,
198                             match=models.MatchValue(value=value)
199                         )
200                     )
201                 elif isinstance(value, dict) and "range" in value:
202                     # 范围过滤
203                     filter_clauses.append(
204                         models.FieldCondition(
```

```

205         key=key,
206         range=models.Range(
207             gte=value["range"].get("gte"),
208             lte=value["range"].get("lte")
209         )
210     )
211 )
212
213 if filter_clauses:
214     search_filter = models.Filter(
215         must=filter_clauses
216     )
217
218 # 执行搜索
219 start_time = time.time()
220
221 try:
222     results = self.client.search(
223         collection_name=collection_name,
224         query_vector=query_vector,
225         query_filter=search_filter,
226         limit=limit,
227         score_threshold=score_threshold,
228         with_payload=True,
229         with_vectors=False # 不返回向量以节省带宽
230     )
231
232     search_time = time.time() - start_time
233
234 # 格式化结果
235 formatted_results = []
236 for result in results:
237     formatted_results.append({
238         "id": result.id,
239         "score": result.score,
240         "text": result.payload.get("text", ""),
241         "metadata": {k: v for k, v in result.payload.items
242 () if k != "text"}
243     })
244
245 print(f"搜索完成, 耗时: {search_time*1000:.1f}ms, 返回 {len
(results)} 个结果")
246 return formatted_results
247
248 except Exception as e:
249     print(f"搜索失败: {e}")
250     return []
251
252 def semantic_chunking_and_index(
253     self,
254     collection_name: str,
255     long_document: str,
256     chunk_size: int = 500,
257     overlap: int = 50,
258     metadata: Dict = None
259 ):
260     """语义分块并索引长文档"""

```

```
260
261     def split_by_sentences(text: str) -> List[str]:
262         """按句子分割文本"""
263         import re
264         sentences = re.split(r'[.!?]+', text)
265         return [s.strip() for s in sentences if s.strip()]
266
267     def create_semantic_chunks(sentences: List[str], target_size:
268 int) -> List[str]:
269         """创建语义相关的分块"""
270         chunks = []
271         current_chunk = ""
272         current_size = 0
273
274         for sentence in sentences:
275             sentence_size = len(sentence)
276
277             if current_size + sentence_size <= target_size:
278                 current_chunk += sentence + ". "
279                 current_size += sentence_size
280             else:
281                 if current_chunk:
282                     chunks.append(current_chunk.strip())
283                     current_chunk = sentence + ". "
284                     current_size = sentence_size
285
286             if current_chunk:
287                 chunks.append(current_chunk.strip())
288
289         return chunks
290
291 # 分解文档
292 sentences = split_by_sentences(long_document)
293 chunks = create_semantic_chunks(sentences, chunk_size)
294
295 print(f"文档分割为 {len(chunks)} 个语义块")
296
297 # 为每个块添加元数据
298 chunk_metadatas = []
299 base_metadata = metadata or {}
300
301 for i, chunk in enumerate(chunks):
302     chunk_meta = {
303         **base_metadata,
304         "chunk_id": i,
305         "total_chunks": len(chunks),
306         "chunk_size": len(chunk),
307         "document_type": "semantic_chunk"
308     }
309     chunk_metadatas.append(chunk_meta)
310
311 # 批量索引
312 self.batch_upsert_documents(
313     collection_name=collection_name,
314     documents=chunks,
315     metadatas=chunk_metadatas
316 )
```

```
316         return len(chunks)
317
318     def analyze_collection_performance(self, collection_name: str):
319         """分析集合性能指标"""
320
321         try:
322             # 获取集合信息
323             collection_info = self.client.get_collection(
324                 collection_name)
325
326             print(f"\n=== 集合 '{collection_name}' 性能分析 ===")
327             print(f"向量数量: {collection_info.points_count:,}")
328             print(f"向量维度: {collection_info.config.params.vectors.
329                 size}")
330             print(f"距离度量: {collection_info.config.params.vectors.
331                 distance}")
332
333             # HNSW配置
334             hnsw = collection_info.config.hnsw_config
335             print(f"\nHNSW配置:")
336             print(f"    M (连接数): {hnsw.m}")
337             print(f"    ef_construct: {hnsw.ef_construct}")
338             print(f"    索引位置: {'磁盘' if hnsw.on_disk else '内存'}")
339
340             # 优化器配置
341             optimizer = collection_info.config.optimizer_config
342             print(f"\n优化器配置:")
343             print(f"    默认段数: {optimizer.default_segment_number}")
344             print(f"    最大段大小: {optimizer.max_segment_size:,}")
345             print(f"    内存映射阈值: {optimizer.memmap_threshold:,}")
346
347             # 执行性能测试
348             self._benchmark_search_performance(collection_name)
349
350         except Exception as e:
351             print(f"性能分析失败: {e}")
352
353     def _benchmark_search_performance(self, collection_name: str,
354         num_queries: int = 100):
355         """基准搜索性能测试"""
356
357         # 生成测试查询
358         test_queries = [
359             "机器学习算法", "深度学习网络", "数据科学分析", "人工智能应用",
360             "自然语言处理", "计算机视觉", "推荐系统", "大数据处理",
361             "云计算平台", "软件工程实践"
362         ] * (num_queries // 10 + 1)
363
364         test_queries = test_queries[:num_queries]
365
366         print(f"\n=== 搜索性能基准测试 ({num_queries} 次查询) ===")
367
368         latencies = []
```

```

367     for i, query in enumerate(test_queries):
368         start_time = time.time()
369
370         try:
371             results = self.hybrid_search(
372                 collection_name=collection_name,
373                 query=query,
374                 limit=10
375             )
376
377             latency = (time.time() - start_time) * 1000 # 转换为毫
秒
378             latencies.append(latency)
379
380             if (i + 1) % 20 == 0:
381                 print(f"已完成 {i + 1}/{num_queries} 次查询")
382
383             except Exception as e:
384                 print(f"查询 {i+1} 失败: {e}")
385
386         if latencies:
387             avg_latency = np.mean(latencies)
388             p50_latency = np.percentile(latencies, 50)
389             p95_latency = np.percentile(latencies, 95)
390             p99_latency = np.percentile(latencies, 99)
391             min_latency = np.min(latencies)
392             max_latency = np.max(latencies)
393
394             print(f"\n性能指标:")
395             print(f"  平均延迟: {avg_latency:.1f}ms")
396             print(f"  P50延迟: {p50_latency:.1f}ms")
397             print(f"  P95延迟: {p95_latency:.1f}ms")
398             print(f"  P99延迟: {p99_latency:.1f}ms")
399             print(f"  最小延迟: {min_latency:.1f}ms")
400             print(f"  最大延迟: {max_latency:.1f}ms")
401             print(f"  QPS估算: {1000/avg_latency:.1f}")
402
403 # 使用示例和演示
404 def demonstrate_qdrant_advanced_features():
405     """演示Qdrant高级特性"""
406
407     # 初始化管理器
408     qdrant_manager = AdvancedQdrantManager()
409
410     collection_name = "advanced_demo"
411
412     print("=== Qdrant高级特性演示 ===\n")
413
414     # 1. 创建优化的集合
415     print("1. 创建优化集合配置...")
416     qdrant_manager.create_optimized_collection(
417         collection_name=collection_name,
418         storage_type="memory"
419     )
420
421     # 2. 准备示例文档
422     documents = [

```



```
423     "深度学习是机器学习的一个分支，它基于人工神经网络进行学习和决
424     策。",
425     "自然语言处理（NLP）是人工智能的一个重要应用领域，专注于让计算
426     机理解和生成人类语言。",
427     "计算机视觉技术能够让机器识别和理解图像中的内容，在自动驾驶、医
428     疗诊断等领域有广泛应用。",
429     "推荐系统通过分析用户行为和偏好，为用户推荐可能感兴趣的商品或内
430     容。",
431     "大数据处理技术能够高效处理和分析海量数据，从中提取有价值的信息
432     和洞察。",
433     "云计算提供了可扩展、按需使用的计算资源，是现代软件开发和部署的
434     重要基础设施。",
435     "机器学习模型需要大量训练数据来学习模式和规律，数据质量直接影响
436     模型性能。",
437     "人工智能伦理关注AI技术的公平性、透明性和责任性，确保AI造福人类
438     社会。",
439     "量子计算利用量子力学原理进行计算，在特定问题上可能实现指数级的
440     性能提升。",
441     "区块链技术提供了去中心化、不可篡改的数据存储方案，在金融、供应
442     链等领域有重要应用。"
443 ]
444
445 # 为文档添加元数据
446 metadatas = []
447 topics = ["深度学习", "NLP", "计算机视觉", "推荐系统", "大数据", "
448 云计算", "机器学习", "AI伦理", "量子计算", "区块链"]
449
450 for i, topic in enumerate(topics):
451     metadatas.append({
452         "topic": topic,
453         "difficulty": "中级" if i % 3 == 0 else "初级",
454         "category": "技术",
455         "word_count": len(documents[i].split())
456     })
457
458 # 3. 批量插入文档
459 print("\n2. 批量插入示例文档...")
460 qdrant_manager.batch_upsert_documents(
461     collection_name=collection_name,
462     documents=documents,
463     metadatas=metadatas
464 )
465
466 # 4. 执行混合搜索
467 print("\n3. 执行混合搜索...")
468
469 # 基础搜索
470 results = qdrant_manager.hybrid_search(
471     collection_name=collection_name,
472     query="人工智能和机器学习",
473     limit=5
474 )
475
476 print("搜索结果:")
477 for i, result in enumerate(results):
```

```
467         print(f"{i+1}. [相似度: {result['score']:.3f}] {result['text'][:50]}...")
468         print(f"    主题: {result['metadata'].get('topic', 'N/A')}")
469
470     # 带过滤条件的搜索
471     print("\n4. 带过滤条件的搜索（只搜索初级难度内容）...")
472     filtered_results = qdrant_manager.hybrid_search(
473         collection_name=collection_name,
474         query="计算机技术",
475         limit=3,
476         filter_conditions={"difficulty": "初级"}
477     )
478
479     print("过滤后的搜索结果:")
480     for i, result in enumerate(filtered_results):
481         print(f"{i+1}. [相似度: {result['score']:.3f}] {result['text'][:50]}...")
482         print(f"    难度: {result['metadata'].get('difficulty', 'N/A')}")
483     )
484
485     # 5. 语义分块示例
486     print("\n5. 语义分块长文档...")
487     long_document = """
488     人工智能（Artificial Intelligence，AI）是计算机科学的一个分支，它旨在创建能够执行通常需要人类智能的任务的系统。
489
490     机器学习是人工智能的一个重要子领域，它使计算机能够在没有明确编程的情况下学习和改进。通过分析大量数据，
491     机器学习算法可以识别模式并做出预测。常见的机器学习类型包括监督学习、无监督学习和强化学习。
492
493     深度学习是机器学习的一个特殊分支，它使用人工神经网络来模拟人类大脑的工作方式。深度学习在图像识别、
494     语音识别和自然语言处理等领域取得了突破性进展。卷积神经网络（CNN）特别适用于图像处理任务，
495     而循环神经网络（RNN）和长短期记忆网络（LSTM）则在序列数据处理方面表现出色。
496
497     自然语言处理（NLP）是人工智能的另一个重要应用领域，它专注于使计算机能够理解、解释和生成人类语言。
498     现代NLP系统使用深度学习技术，如Transformer架构，来实现机器翻译、文本摘要、情感分析等功能。
499
500     人工智能的应用正在快速扩展到各个行业，包括医疗保健、金融、交通、教育和娱乐。然而，随着AI技术的发展，
501     也出现了关于隐私、就业影响和算法偏见等伦理问题的讨论。
502     """
503
504     chunks_count = qdrant_manager.semantic_chunking_and_index(
505         collection_name=collection_name,
506         long_document=long_document,
507         chunk_size=300,
508         metadata={"document_type": "教学材料", "subject": "人工智能概述"}
509     )
```

```
509
510     print(f"长文档分割为 {chunks_count} 个语义块并已索引")
511
512     # 6. 搜索分块内容
513     print("\n6. 搜索分块内容...")
514     chunk_results = qdrant_manager.hybrid_search(
515         collection_name=collection_name,
516         query="深度学习神经网络",
517         limit=3,
518         filter_conditions={"document_type": "教学材料"}
519     )
520
521     print("分块搜索结果:")
522     for i, result in enumerate(chunk_results):
523         print(f"{i+1}. [相似度: {result['score']:.3f}]")
524         print(f"    内容: {result['text'][:100]}...")
525         print(f"    块ID: {result['metadata'].get('chunk_id', 'N/A')}")
526
527     # 7. 性能分析
528     print("\n7. 集合性能分析...")
529     qdrant_manager.analyze_collection_performance(collection_name)
530
531     print("\n=== 演示完成 ===")
532
533 # 运行演示
534 # demonstrate_qdrant_advanced_features()
```

Listing 8: Qdrant 高级配置和使用

7 检索增强生成 (RAG) 系统

RAG 系统结合了信息检索和生成模型的优势，能够基于外部知识库生成准确、时效性强的回答。

7.1 RAG 系统架构

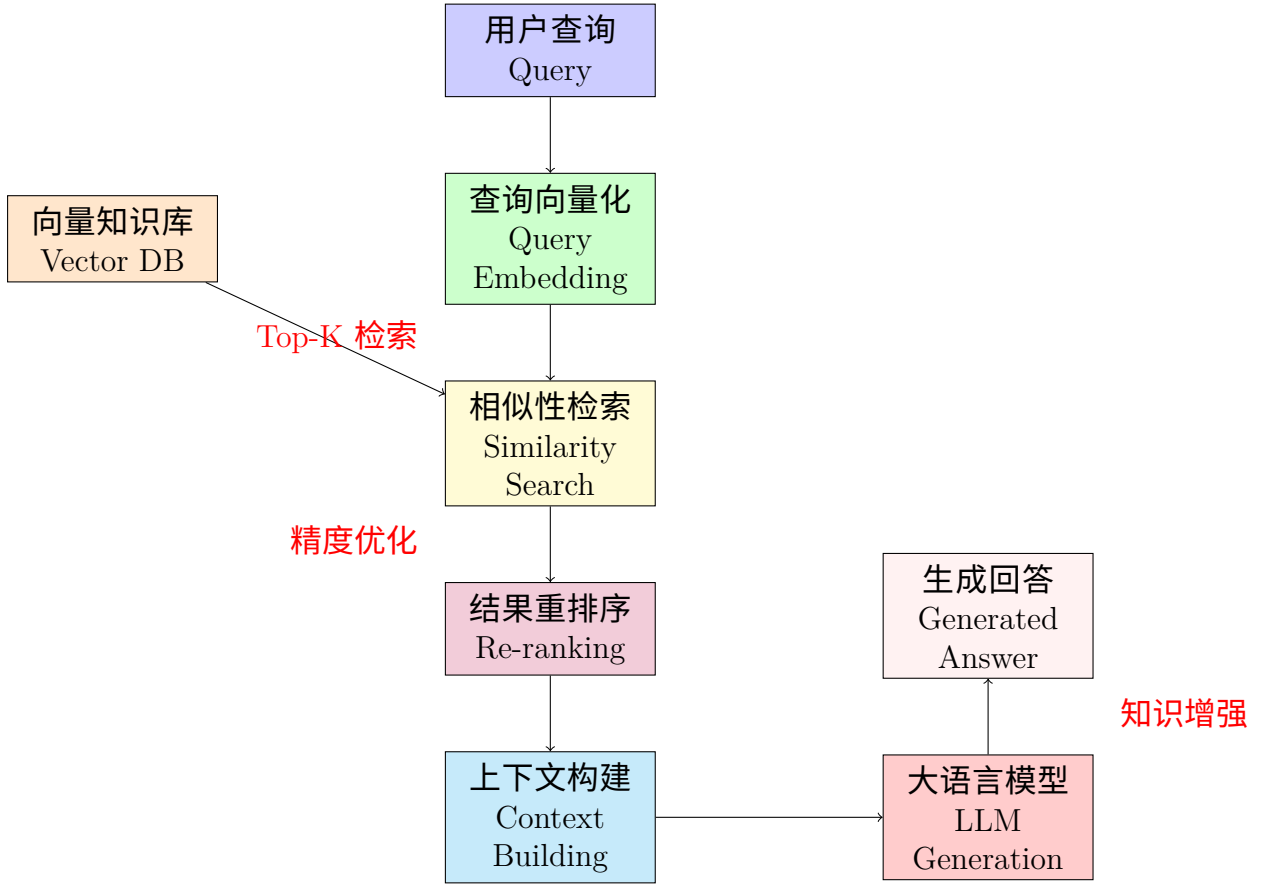


Figure 5: RAG 系统架构流程

7.2 RAG 核心数学原理

RAG 系统的核心是计算查询与文档的相关性，并基于检索结果生成回答。

7.2.1 检索相关性计算

给定查询 q 和文档集合 $\{d_1, d_2, \dots, d_n\}$ ，相关性分数计算为：

$$\text{relevance}(q, d_i) = \text{similarity}(\text{embed}(q), \text{embed}(d_i)) \quad (10)$$

其中 $\text{embed}(\cdot)$ 是文本向量化函数。

7.2.2 概率生成模型

RAG 的生成概率可以表示为：

$$P(y|x) = \sum_{i=1}^k P(y|x, d_i) \cdot P(d_i|x) \quad (11)$$

$$\text{其中：} \quad P(d_i|x) = \frac{\exp(\text{relevance}(x, d_i))}{\sum_{j=1}^n \exp(\text{relevance}(x, d_j))} \quad (12)$$

参数说明：

- x : 输入查询
- y : 生成的回答
- d_i : 第 i 个检索到的文档
- k : 检索的文档数量

7.3 高级 RAG 实现

```

1 import openai
2 from sentence_transformers import SentenceTransformer, CrossEncoder
3 from typing import List, Dict, Any, Tuple
4 import numpy as np
5 import re
6 from dataclasses import dataclass
7 from abc import ABC, abstractmethod
8
9 @dataclass
10 class Document:
11     """文档数据结构"""
12     id: str
13     content: str
14     metadata: Dict[str, Any]
15     embedding: np.ndarray = None
16
17 @dataclass
18 class RetrievalResult:
19     """检索结果数据结构"""
20     document: Document
21     score: float
22     rank: int
23
24 class DocumentProcessor(ABC):
25     """文档处理器抽象基类"""
26
27     @abstractmethod
28     def chunk_document(self, text: str, metadata: Dict = None) -> List[
        Document]:
29         """将文档分块"""
30         pass
31
32     @abstractmethod
33     def preprocess_text(self, text: str) -> str:
34         """文本预处理"""
35         pass
36
37 class SemanticDocumentProcessor(DocumentProcessor):
38     """语义文档处理器"""
39
40     def __init__(self, chunk_size: int = 512, overlap: int = 50):
41         self.chunk_size = chunk_size
42         self.overlap = overlap
43

```

```

44     def preprocess_text(self, text: str) -> str:
45         """清理和标准化文本"""
46         # 移除多余空白
47         text = re.sub(r'\s+', ' ', text)
48
49         # 移除特殊字符（保留中文、英文、数字和基本标点）
50         text = re.sub(r'[^\\w\\s\\u4e00-\\u9fff.,;:!?()-]', '', text)
51
52         # 标准化标点符号
53         text = text.replace(', ', ',').replace('。 ', '.').replace('! ', '! ',
54         '!').replace('? ', '?')
55
56         return text.strip()
57
58     def chunk_document(self, text: str, metadata: Dict = None) -> List[
59         Document]:
60         """智能文档分块"""
61         if metadata is None:
62             metadata = {}
63
64         # 预处理文本
65         clean_text = self.preprocess_text(text)
66
67         # 按段落分割
68         paragraphs = [p.strip() for p in clean_text.split('\\n\\n') if p.
69         strip()]
70
71         chunks = []
72         current_chunk = ""
73         current_length = 0
74         chunk_id = 0
75
76         for paragraph in paragraphs:
77             paragraph_length = len(paragraph)
78
79             # 如果当前块加上新段落不超过限制，则添加
80             if current_length + paragraph_length <= self.chunk_size:
81                 current_chunk += paragraph + "\\n\\n"
82                 current_length += paragraph_length + 2 # +2 for
83                 newlines
84             else:
85                 # 保存当前块
86                 if current_chunk.strip():
87                     chunks.append(Document(
88                         id=f"chunk_{chunk_id}",
89                         content=current_chunk.strip(),
90                         metadata={
91                             **metadata,
92                             "chunk_id": chunk_id,
93                             "chunk_size": current_length,
94                             "chunk_type": "semantic"
95                         })
96                 ))
97                 chunk_id += 1
98
99             # 开始新块
100             current_chunk = paragraph + "\\n\\n"

```

```

97         current_length = paragraph_length + 2
98
99     # 添加最后一个块
100     if current_chunk.strip():
101         chunks.append(Document(
102             id=f"chunk_{chunk_id}",
103             content=current_chunk.strip(),
104             metadata={
105                 **metadata,
106                 "chunk_id": chunk_id,
107                 "chunk_size": current_length,
108                 "chunk_type": "semantic"
109             })
110     ))
111
112     return chunks
113
114 class HybridRetriever:
115     """混合检索器：结合稠密和稀疏检索"""
116
117     def __init__(self,
118                 embedding_model: str = "all-MiniLM-L6-v2",
119                 cross_encoder_model: str = "cross-encoder/ms-marco-
120 MiniLM-L-6-v2"):
121
122         # 稠密检索：向量相似性
123         self.embedding_model = SentenceTransformer(embedding_model)
124
125         # 重排序：交叉编码器
126         self.cross_encoder = CrossEncoder(cross_encoder_model)
127
128         # 文档存储
129         self.documents: List[Document] = []
130         self.document_embeddings: np.ndarray = None
131
132     def add_documents(self, documents: List[Document]):
133         """添加文档到检索器"""
134         self.documents.extend(documents)
135
136         # 计算文档嵌入
137         texts = [doc.content for doc in documents]
138         embeddings = self.embedding_model.encode(texts,
139 convert_to_numpy=True)
140
141         if self.document_embeddings is None:
142             self.document_embeddings = embeddings
143         else:
144             self.document_embeddings = np.vstack([self.
145 document_embeddings, embeddings])
146
147         # 更新文档对象的嵌入
148         start_idx = len(self.documents) - len(documents)
149         for i, doc in enumerate(documents):
150             doc.embedding = embeddings[i]
151
152     def dense_retrieval(self, query: str, top_k: int = 20) -> List[
153 RetrievalResult]:

```

```
150     """稠密向量检索"""
151     if not self.documents:
152         return []
153
154     # 编码查询
155     query_embedding = self.embedding_model.encode([query],
156     convert_to_numpy=True)[0]
157
158     # 计算相似度
159     similarities = np.dot(self.document_embeddings, query_embedding
160     )
161
162     # 获取top-k结果
163     top_indices = np.argsort(similarities)[-top_k:][::-1]
164
165     results = []
166     for rank, idx in enumerate(top_indices):
167         results.append(RetrievalResult(
168             document=self.documents[idx],
169             score=float(similarities[idx]),
170             rank=rank
171         ))
172
173     return results
174
175     def sparse_retrieval(self, query: str, documents: List[Document],
176     top_k: int = 20) -> List[RetrievalResult]:
177         """稀疏检索（基于BM25）"""
178
179         # 简化的TF-IDF相似度计算
180         from collections import Counter
181         import math
182
183         # 分词（简化版本）
184         def tokenize(text: str) -> List[str]:
185             return re.findall(r'\b\w+\b', text.lower())
186
187         query_tokens = tokenize(query)
188         doc_tokens_list = [tokenize(doc.content) for doc in documents]
189
190         # 计算文档频率
191         doc_freq = Counter()
192         for doc_tokens in doc_tokens_list:
193             for token in set(doc_tokens):
194                 doc_freq[token] += 1
195
196         # 计算BM25分数
197         k1, b = 1.2, 0.75
198         avg_doc_len = np.mean([len(tokens) for tokens in
199         doc_tokens_list])
200
201         scores = []
202         for doc_tokens in doc_tokens_list:
203             doc_len = len(doc_tokens)
204             doc_counter = Counter(doc_tokens)
205             score = 0
206
207             for token in query_tokens:
```



```

203         if token in doc_counter:
204             tf = doc_counter[token]
205             idf = math.log((len(documents) - doc_freq[token] +
0.5) / (doc_freq[token] + 0.5))
206
207             score += idf * (tf * (k1 + 1)) / (tf + k1 * (1 - b
+ b * doc_len / avg_doc_len))
208
209             scores.append(score)
210
211     # 获取top-k结果
212     top_indices = np.argsort(scores)[-top_k:][::-1]
213
214     results = []
215     for rank, idx in enumerate(top_indices):
216         results.append(RetrievalResult(
217             document=documents[idx],
218             score=scores[idx],
219             rank=rank
220         ))
221
222     return results
223
224     def hybrid_search(self, query: str,
225                       dense_weight: float = 0.7,
226                       sparse_weight: float = 0.3,
227                       top_k: int = 10) -> List[RetrievalResult]:
228         """混合搜索：结合稠密和稀疏检索"""
229
230     # 稠密检索
231     dense_results = self.dense_retrieval(query, top_k=min(50, len(
self.documents)))
232
233     # 稀疏检索
234     sparse_results = self.sparse_retrieval(query, self.documents,
top_k=min(50, len(self.documents)))
235
236     # 分数归一化和融合
237     def normalize_scores(results: List[RetrievalResult]) -> List[
RetrievalResult]:
238         if not results:
239             return results
240
241         scores = [r.score for r in results]
242         min_score, max_score = min(scores), max(scores)
243
244         if max_score == min_score:
245             return results
246
247         for result in results:
248             result.score = (result.score - min_score) / (max_score
- min_score)
249
250         return results
251
252     # 归一化分数
253     dense_results = normalize_scores(dense_results)

```

```
254     sparse_results = normalize_scores(sparse_results)
255
256     # 创建文档ID到分数的映射
257     dense_scores = {r.document.id: r.score for r in dense_results}
258     sparse_scores = {r.document.id: r.score for r in sparse_results}
259 }
260
261 # 融合分数
262 all_doc_ids = set(dense_scores.keys()) | set(sparse_scores.keys())
263
264 hybrid_scores = {}
265
266 for doc_id in all_doc_ids:
267     d_score = dense_scores.get(doc_id, 0)
268     s_score = sparse_scores.get(doc_id, 0)
269     hybrid_scores[doc_id] = dense_weight * d_score +
270 sparse_weight * s_score
271
272 # 创建最终结果
273 doc_id_to_doc = {doc.id: doc for doc in self.documents}
274 final_results = []
275
276 for doc_id, score in sorted(hybrid_scores.items(), key=lambda x
277 : x[1], reverse=True)[:top_k]:
278     final_results.append(RetrievalResult(
279         document=doc_id_to_doc[doc_id],
280         score=score,
281         rank=len(final_results)
282     ))
283
284 return final_results
285
286 def rerank_with_cross_encoder(self, query: str,
287                               retrieval_results: List[
288 RetrievalResult],
289                               top_k: int = 10) -> List[
290 RetrievalResult]:
291     """使用交叉编码器重排序"""
292
293     if len(retrieval_results) <= top_k:
294         return retrieval_results
295
296     # 准备查询-文档对
297     query_doc_pairs = [(query, result.document.content) for result
298 in retrieval_results]
299
300     # 计算交叉编码器分数
301     cross_scores = self.cross_encoder.predict(query_doc_pairs)
302
303     # 更新分数并重新排序
304     for i, result in enumerate(retrieval_results):
305         result.score = float(cross_scores[i])
306
307     # 按新分数排序
308     reranked_results = sorted(retrieval_results, key=lambda x: x.
309 score, reverse=True)
```

```
303     # 更新排名
304     for rank, result in enumerate(reranked_results[:top_k]):
305         result.rank = rank
306
307     return reranked_results[:top_k]
308
309 class AdvancedRAGSystem:
310     """高级RAG系统"""
311
312     def __init__(self, openai_api_key: str):
313         # 设置OpenAI API
314         openai.api_key = openai_api_key
315
316     # 初始化组件
317     self.document_processor = SemanticDocumentProcessor()
318     self.retriever = HybridRetriever()
319
320     # 生成配置
321     self.generation_config = {
322         "model": "gpt-3.5-turbo",
323         "temperature": 0.1,
324         "max_tokens": 1000,
325         "top_p": 0.9
326     }
327
328     def add_documents_to_knowledge_base(self, texts: List[str],
329                                       metadatas: List[Dict] = None):
330         """添加文档到知识库"""
331         if metadatas is None:
332             metadatas = [{}] * len(texts)
333
334         all_chunks = []
335
336         for text, metadata in zip(texts, metadatas):
337             # 文档分块
338             chunks = self.document_processor.chunk_document(text,
339                                                             metadata)
340             all_chunks.extend(chunks)
341
342         # 添加到检索器
343         self.retriever.add_documents(all_chunks)
344
345         print(f"已添加 {len(all_chunks)} 个文档块到知识库")
346
347     def generate_answer(self, query: str,
348                        context_documents: List[Document],
349                        max_context_length: int = 3000) -> str:
350         """基于上下文生成答案"""
351
352         # 构建上下文
353         context_parts = []
354         current_length = 0
355
356         for doc in context_documents:
357             doc_text = f"文档 {doc.id}:\n{doc.content}\n"
```

```

358         if current_length + doc_length <= max_context_length:
359             context_parts.append(doc_text)
360             current_length += doc_length
361         else:
362             break
363
364     context = "\n".join(context_parts)
365
366     # 构建提示
367     system_prompt = """你是一个有用的AI助手。请根据提供的上下文文档
    回答用户的问题。
368
369 要求：
370 1. 答案必须基于提供的上下文内容
371 2. 如果上下文中没有相关信息，请明确说明
372 3. 引用具体的文档内容支持你的答案
373 4. 保持回答的准确性和客观性
374 5. 如果问题涉及多个方面，请全面回答"""
375
376     user_prompt = f"""上下文文档：
377 {context}
378
379 问题：{query}
380
381 请基于上述文档内容回答问题："""
382
383     # 调用LLM生成答案
384     try:
385         response = openai.ChatCompletion.create(
386             model=self.generation_config["model"],
387             messages=[
388                 {"role": "system", "content": system_prompt},
389                 {"role": "user", "content": user_prompt}
390             ],
391             temperature=self.generation_config["temperature"],
392             max_tokens=self.generation_config["max_tokens"],
393             top_p=self.generation_config["top_p"]
394         )
395
396         answer = response.choices[0].message.content.strip()
397         return answer
398
399     except Exception as e:
400         return f"生成答案时出现错误：{str(e)}"
401
402     def query(self, question: str,
403               retrieval_top_k: int = 20,
404               final_top_k: int = 5,
405               use_reranking: bool = True) -> Dict[str, Any]:
406         """执行RAG查询"""
407
408         print(f"处理查询：{question}")
409
410         # 1. 混合检索
411         print("正在执行混合检索...")
412         retrieval_results = self.retriever.hybrid_search(

```

```
413         query=question,
414         top_k=retrieval_top_k
415     )
416
417     if not retrieval_results:
418         return {
419             "answer": "抱歉，我在知识库中没有找到相关信息来回答您的
420 问题。",
421             "sources": [],
422             "retrieval_scores": []
423         }
424
425     # 2. 重排序 (可选)
426     if use_reranking and len(retrieval_results) > final_top_k:
427         print("正在使用交叉编码器重排序...")
428         retrieval_results = self.retriever.
429         rerank_with_cross_encoder(
430             query=question,
431             retrieval_results=retrieval_results,
432             top_k=final_top_k
433         )
434     else:
435         retrieval_results = retrieval_results[:final_top_k]
436
437     # 3. 生成答案
438     print("正在生成答案...")
439     context_docs = [result.document for result in retrieval_results
440 ]
441     answer = self.generate_answer(question, context_docs)
442
443     # 4. 整理结果
444     sources = []
445     for result in retrieval_results:
446         sources.append({
447             "id": result.document.id,
448             "content": result.document.content[:200] + "..." if len
449 (result.document.content) > 200 else result.document.content,
450             "metadata": result.document.metadata,
451             "score": result.score,
452             "rank": result.rank
453         })
454
455     return {
456         "answer": answer,
457         "sources": sources,
458         "retrieval_scores": [r.score for r in retrieval_results]
459     }
460
461 # 使用示例和演示
462 def demonstrate_advanced_rag():
463     """演示高级RAG系统"""
464
465     # 注意：需要提供真实的OpenAI API密钥
466     rag_system = AdvancedRAGSystem("your-openai-api-key")
467
468     print("=== 高级RAG系统演示 ===\n")
```

1. 准备知识库文档

```
knowledge_documents = [
```

```
    """
```

深度学习是机器学习的一个分支，它基于人工神经网络进行学习和决策。深度学习的"深度"指的是神经网络具有多个隐藏层。

典型的深度学习模型包括卷积神经网络（CNN）、循环神经网络（RNN）和Transformer。

卷积神经网络特别适用于图像处理任务，因为它能够有效地提取图像的空间特征。CNN通过卷积层、池化层和全连接层的组合来识别图像中的模式和特征。

循环神经网络则擅长处理序列数据，如文本和时间序列。RNN的变种包括长短期记忆网络（LSTM）和门控循环单元（GRU），它们能够更好地处理长序列数据。

```
    """,
```

```
    """
```

自然语言处理（NLP）是人工智能的一个重要分支，专注于让计算机理解和生成人类语言。现代NLP技术主要基于深度学习方法。

Transformer架构是现代NLP的基础，它引入了注意力机制，能够更好地理解文本中单词之间的关系。BERT、GPT等著名模型都基于Transformer架构。

常见的NLP任务包括文本分类、命名实体识别、机器翻译、文本摘要和情感分析。这些任务都可以通过预训练的语言模型进行微调来实现。

预训练语言模型的训练过程通常包括两个阶段：预训练阶段使用大量无标签文本进行自监督学习，微调阶段使用特定任务的有标签数据进行监督学习。

```
    """,
```

```
    """
```

机器学习可以分为三大类：监督学习、无监督学习和强化学习。

监督学习使用有标签的训练数据来学习从输入到输出的映射关系。常见的监督学习算法包括线性回归、逻辑回归、决策树、随机森林和支持向量机。

无监督学习处理没有标签的数据，目标是发现数据中的隐藏模式。主要技术包括聚类算法（如K-means、层次聚类）和降维算法（如PCA、t-SNE）。

强化学习通过与环境交互来学习最优策略，代理（agent）通过试错来最大化累积奖励。著名的强化学习算法包括Q-learning、策略梯度和Actor-Critic方法。

特征工程在传统机器学习中非常重要，包括特征选择、特征提取和特征构造。好的特征工程往往比复杂的算法更能提升模型性能。

```
    """,
```

```
    """
```

计算机视觉是人工智能的一个重要应用领域，目标是让计算机能够理解和解释视觉信息。

图像分类是计算机视觉的基础任务，目标是将图像分配到预定义类别中。ResNet、VGG、Inception等深度学习模型在图像分类任务上取得了突破性

进展。

目标检测不仅要识别图像中的对象，还要确定它们的位置。YOLO、R-CNN 系列和SSD是流行的目标检测算法。

语义分割将图像中的每个像素分配到特定的类别，实现像素级的分类。FCN、U-Net和DeepLab是常用的语义分割方法。

计算机视觉在自动驾驶、医疗影像分析、安防监控和增强现实等领域有广泛应用。

```
"""  
]  
  
# 文档元数据  
metadatas = [  
    {"topic": "深度学习", "difficulty": "中级", "field": "机器学习"},  
    {"topic": "自然语言处理", "difficulty": "中级", "field": "人工智能"},  
    {"topic": "机器学习基础", "difficulty": "初级", "field": "机器学习"},  
    {"topic": "计算机视觉", "difficulty": "中级", "field": "人工智能"}  
]
```

2. 构建知识库

```
print("1. 构建知识库...")  
rag_system.add_documents_to_knowledge_base(knowledge_documents,  
metadatas)
```

3. 测试查询

```
test_queries = [  
    "什么是深度学习？它包含哪些主要的神经网络类型？",  
    "Transformer架构在自然语言处理中的作用是什么？",  
    "监督学习和无监督学习有什么区别？",  
    "计算机视觉中的目标检测任务是如何工作的？",  
    "机器学习中特征工程的重要性如何？"  
]
```

```
print(f"\n2. 执行 {len(test_queries)} 个测试查询...\n")
```

```
for i, query in enumerate(test_queries, 1):  
    print(f"=== 查询 {i} ===")  
    print(f"问题: {query}")
```

执行RAG查询

```
result = rag_system.query(  
    question=query,  
    retrieval_top_k=10,  
    final_top_k=3,  
    use_reranking=True  
)
```

```
print(f"\n答案: {result['answer']}")
```

```
549     print(f"\n相关文档源 ({len(result['sources'])}):")
550     for j, source in enumerate(result['sources']):
551         print(f"{j+1}. [分数: {source['score']:.3f}] {source['content']}")
552         print(f"    主题: {source['metadata'].get('topic', 'N/A')}")
553
554     print(f"\n检索分数: {[f'{score:.3f}' for score in result['retrieval_scores']]}")
555     print("-" * 80 + "\n")
556
557     print("=== RAG系统演示完成 ===")
558
559 # 运行演示
560 # demonstrate_advanced_rag()
```

Listing 9: 高级 RAG 系统实现

8 性能优化与评估

8.1 系统性能指标

在 AI/LLM 和向量系统中，关键性能指标包括：

Table 3: 性能评估指标体系			
类别	指标	计算公式	目标值
检索质量	召回率	$R = \frac{ \text{检索相关} \cap \text{实际相关} }{ \text{实际相关} }$	> 0.8
	准确率	$P = \frac{ \text{检索相关} \cap \text{实际相关} }{ \text{检索相关} }$	> 0.7
	F1 分数	$F1 = 2 \times \frac{P \times R}{P + R}$	> 0.75
系统性能	查询延迟	平均响应时间	< 200ms
	吞吐量	QPS	> 100
	P99 延迟	99% 分位延迟	< 500ms
资源使用	内存使用	峰值内存/总内存	< 80%
	CPU 使用	平均 CPU 使用率	< 70%

8.2 优化策略

8.2.1 索引优化

Algorithm 1 HNSW 参数自动调优算法

```

1: procedure AUTOTUNEHNSW(queries, ground_truth, data_size)
2:   best_params  $\leftarrow \emptyset$ 
3:   best_score  $\leftarrow 0$ 
4:   for  $M \in \{8, 16, 32\}$  do
5:     for  $ef\_construct \in \{50, 100, 200\}$  do
6:       for  $ef\_search \in \{10, 50, 100\}$  do
7:         index  $\leftarrow$  BuildHNSW( $M, ef\_construct$ )
8:         results  $\leftarrow$  Search(index, queries,  $ef\_search$ )
9:         recall  $\leftarrow$  CalculateRecall(results, ground_truth)
10:        latency  $\leftarrow$  MeasureLatency(index, queries)
11:        score  $\leftarrow 0.7 \times recall - 0.3 \times latency$ 
12:        if score > best_score then
13:          best_score  $\leftarrow$  score
14:          best_params  $\leftarrow (M, ef\_construct, ef\_search)$ 
15:        end if
16:      end for
17:    end for
18:  end for
19:  return best_params
20: end procedure

```

8.2.2 缓存策略

```

1 import redis
2 import hashlib
3 import pickle
4 from typing import Any, Optional
5 from dataclasses import dataclass
6 from abc import ABC, abstractmethod
7
8 @dataclass
9 class CacheItem:
10     """缓存项"""
11     key: str
12     value: Any
13     ttl: int # 生存时间 (秒)
14     size: int # 大小 (字节)
15
16 class CacheStrategy(ABC):
17     """缓存策略抽象基类"""
18
19     @abstractmethod
20     def should_cache(self, key: str, value: Any, cost: float) -> bool:
21         """判断是否应该缓存"""
22         pass
23

```

```

24     @abstractmethod
25     def get_ttl(self, key: str, value: Any) -> int:
26         """获取缓存TTL"""
27         pass
28
29 class AdaptiveCacheStrategy(CacheStrategy):
30     """自适应缓存策略"""
31
32     def __init__(self, base_ttl: int = 3600):
33         self.base_ttl = base_ttl
34         self.access_counts = {}
35         self.cost_history = {}
36
37     def should_cache(self, key: str, value: Any, cost: float) -> bool:
38         """基于成本和访问频率决定是否缓存"""
39         # 高成本操作总是缓存
40         if cost > 1.0: # 超过1秒
41             return True
42
43         # 基于历史访问频率决定
44         access_count = self.access_counts.get(key, 0)
45         return access_count >= 2 or cost > 0.1
46
47     def get_ttl(self, key: str, value: Any) -> int:
48         """基于访问频率和成本调整TTL"""
49         access_count = self.access_counts.get(key, 0)
50         cost = self.cost_history.get(key, 0)
51
52         # 高频访问和高成本的数据缓存更久
53         multiplier = 1 + min(access_count * 0.1, 2.0) + min(cost, 2.0)
54         return int(self.base_ttl * multiplier)
55
56     def record_access(self, key: str, cost: float = 0):
57         """记录访问"""
58         self.access_counts[key] = self.access_counts.get(key, 0) + 1
59         if cost > 0:
60             self.cost_history[key] = cost
61
62 class MultiLevelCache:
63     """多级缓存系统"""
64
65     def __init__(self,
66                 l1_size: int = 1000, # L1缓存大小 (内存)
67                 redis_client=None, # L2缓存 (Redis)
68                 strategy: CacheStrategy = None):
69
70         # L1缓存: 进程内存缓存
71         self.l1_cache = {}
72         self.l1_size = l1_size
73         self.l1_access_order = [] # LRU顺序
74
75         # L2缓存: Redis缓存
76         self.redis_client = redis_client or redis.Redis(host='localhost', port=6379, db=0)
77
78         # 缓存策略
79         self.strategy = strategy or AdaptiveCacheStrategy()

```

```
80
81     # 统计信息
82     self.stats = {
83         'l1_hits': 0,
84         'l2_hits': 0,
85         'misses': 0,
86         'total_requests': 0
87     }
88
89     def _make_key(self, prefix: str, params: dict) -> str:
90         """生成缓存键"""
91         # 创建参数的哈希值以确保键的唯一性
92         params_str = str(sorted(params.items()))
93         hash_obj = hashlib.md5(params_str.encode())
94         return f"{prefix}:{hash_obj.hexdigest()}"
95
96     def _serialize_value(self, value: Any) -> bytes:
97         """序列化值"""
98         return pickle.dumps(value)
99
100    def _deserialize_value(self, data: bytes) -> Any:
101        """反序列化值"""
102        return pickle.loads(data)
103
104    def _evict_l1_if_needed(self):
105        """L1缓存LRU淘汰"""
106        while len(self.l1_cache) >= self.l1_size:
107            oldest_key = self.l1_access_order.pop(0)
108            del self.l1_cache[oldest_key]
109
110    def get(self, prefix: str, params: dict) -> Optional[Any]:
111        """获取缓存值"""
112        key = self._make_key(prefix, params)
113        self.stats['total_requests'] += 1
114
115        # 尝试L1缓存
116        if key in self.l1_cache:
117            # 更新LRU顺序
118            self.l1_access_order.remove(key)
119            self.l1_access_order.append(key)
120
121            self.stats['l1_hits'] += 1
122            return self.l1_cache[key]
123
124        # 尝试L2缓存 (Redis)
125        try:
126            cached_data = self.redis_client.get(key)
127            if cached_data:
128                value = self._deserialize_value(cached_data)
129
130                # 将热点数据提升到L1缓存
131                self._evict_l1_if_needed()
132                self.l1_cache[key] = value
133                self.l1_access_order.append(key)
134
135                self.stats['l2_hits'] += 1
136                return value
```

```

137     except Exception as e:
138         print(f"Redis缓存读取失败: {e}")
139
140     # 缓存未命中
141     self.stats['misses'] += 1
142     return None
143
144     def set(self, prefix: str, params: dict, value: Any,
145             computation_cost: float = 0):
146         """设置缓存值"""
147         key = self._make_key(prefix, params)
148
149         # 记录访问用于策略优化
150         if hasattr(self.strategy, 'record_access'):
151             self.strategy.record_access(key, computation_cost)
152
153         # 判断是否应该缓存
154         if not self.strategy.should_cache(key, value, computation_cost):
155             return
156
157         # 获取TTL
158         ttl = self.strategy.get_ttl(key, value)
159
160         # 存储到L1缓存
161         self._evict_l1_if_needed()
162         self.l1_cache[key] = value
163         self.l1_access_order.append(key)
164
165         # 存储到L2缓存 (Redis)
166         try:
167             serialized_value = self._serialize_value(value)
168             self.redis_client.setex(key, ttl, serialized_value)
169         except Exception as e:
170             print(f"Redis缓存写入失败: {e}")
171
172     def get_stats(self) -> dict:
173         """获取缓存统计信息"""
174         total = self.stats['total_requests']
175         if total == 0:
176             return self.stats
177
178         return {
179             **self.stats,
180             'l1_hit_rate': self.stats['l1_hits'] / total,
181             'l2_hit_rate': self.stats['l2_hits'] / total,
182             'overall_hit_rate': (self.stats['l1_hits'] + self.stats['l2_hits']) / total,
183             'miss_rate': self.stats['misses'] / total
184         }
185
186     def clear(self):
187         """清空缓存"""
188         self.l1_cache.clear()
189         self.l1_access_order.clear()
190         try:
191             self.redis_client.flushdb()

```

```

191         except:
192             pass
193
194 # 带缓存的向量检索系统
195 class CachedVectorSearch:
196     """带缓存的向量搜索系统"""
197
198     def __init__(self, qdrant_client, cache_system: MultiLevelCache):
199         self.client = qdrant_client
200         self.cache = cache_system
201         self.encoder = SentenceTransformer('all-MiniLM-L6-v2')
202
203     def search_with_cache(self,
204                           collection_name: str,
205                           query: str,
206                           limit: int = 10,
207                           score_threshold: float = 0.0) -> List[dict]:
208         """带缓存的向量搜索"""
209         import time
210
211         # 构建缓存参数
212         cache_params = {
213             'collection': collection_name,
214             'query_hash': hashlib.md5(query.encode()).hexdigest(),
215             'limit': limit,
216             'threshold': score_threshold
217         }
218
219         # 尝试从缓存获取
220         cached_result = self.cache.get('vector_search', cache_params)
221         if cached_result is not None:
222             return cached_result
223
224         # 执行实际搜索
225         start_time = time.time()
226
227         query_vector = self.encoder.encode([query])[0].tolist()
228
229         results = self.client.search(
230             collection_name=collection_name,
231             query_vector=query_vector,
232             limit=limit,
233             score_threshold=score_threshold,
234             with_payload=True
235         )
236
237         # 格式化结果
238         formatted_results = []
239         for result in results:
240             formatted_results.append({
241                 'id': result.id,
242                 'score': result.score,
243                 'payload': result.payload
244             })
245
246         computation_cost = time.time() - start_time
247

```

```
248     # 存储到缓存
249     self.cache.set('vector_search', cache_params, formatted_results
250                    , computation_cost)
251
252     return formatted_results
253
254 # 性能监控系统
255 class PerformanceMonitor:
256     """性能监控系统"""
257
258     def __init__(self):
259         self.metrics = {
260             'query_latencies': [],
261             'throughput_samples': [],
262             'error_counts': {},
263             'cache_hit_rates': []
264         }
265         self.start_time = time.time()
266
267     def record_query_latency(self, latency_ms: float):
268         """记录查询延迟"""
269         self.metrics['query_latencies'].append(latency_ms)
270
271     def record_throughput_sample(self, qps: float):
272         """记录吞吐量样本"""
273         self.metrics['throughput_samples'].append(qps)
274
275     def record_error(self, error_type: str):
276         """记录错误"""
277         self.metrics['error_counts'][error_type] = self.metrics['
278 error_counts'].get(error_type, 0) + 1
279
280     def record_cache_hit_rate(self, hit_rate: float):
281         """记录缓存命中率"""
282         self.metrics['cache_hit_rates'].append(hit_rate)
283
284     def get_performance_report(self) -> dict:
285         """生成性能报告"""
286         latencies = self.metrics['query_latencies']
287         throughput = self.metrics['throughput_samples']
288
289         if not latencies:
290             return {"error": "No performance data available"}
291
292         report = {
293             "query_performance": {
294                 "total_queries": len(latencies),
295                 "avg_latency_ms": np.mean(latencies),
296                 "p50_latency_ms": np.percentile(latencies, 50),
297                 "p95_latency_ms": np.percentile(latencies, 95),
298                 "p99_latency_ms": np.percentile(latencies, 99),
299                 "min_latency_ms": np.min(latencies),
300                 "max_latency_ms": np.max(latencies)
301             },
302             "throughput": {
303                 "avg_qps": np.mean(throughput) if throughput else 0,
304                 "peak_qps": np.max(throughput) if throughput else 0
```

```

303         },
304         "reliability": {
305             "error_rate": sum(self.metrics['error_counts'].values()
306 ) / max(len(latencies), 1),
307             "error_breakdown": self.metrics['error_counts']
308         },
309         "caching": {
310             "avg_hit_rate": np.mean(self.metrics['cache_hit_rates'
311 ]) if self.metrics['cache_hit_rates'] else 0
312         },
313         "uptime_hours": (time.time() - self.start_time) / 3600
314     }
315
316     return report
317
318 def print_performance_summary(self):
319     """打印性能摘要"""
320     report = self.get_performance_report()
321
322     if "error" in report:
323         print(report["error"])
324         return
325
326     print("=== 性能监控报告 ===")
327
328     qp = report["query_performance"]
329     print(f"\n查询性能:")
330     print(f"    总查询数: {qp['total_queries']}")
331     print(f"    平均延迟: {qp['avg_latency_ms']:.1f}ms")
332     print(f"    P95延迟: {qp['p95_latency_ms']:.1f}ms")
333     print(f"    P99延迟: {qp['p99_latency_ms']:.1f}ms")
334
335     tp = report["throughput"]
336     print(f"\n吞吐量:")
337     print(f"    平均QPS: {tp['avg_qps']:.1f}")
338     print(f"    峰值QPS: {tp['peak_qps']:.1f}")
339
340     rel = report["reliability"]
341     print(f"\n可靠性:")
342     print(f"    错误率: {rel['error_rate']:.3%}")
343     if rel['error_breakdown']:
344         for error_type, count in rel['error_breakdown'].items():
345             print(f"    {error_type}: {count}")
346
347     cache = report["caching"]
348     print(f"\n缓存:")
349     print(f"    平均命中率: {cache['avg_hit_rate']:.3%}")
350
351     print(f"\n运行时间: {report['uptime_hours']:.1f}小时")

```

Listing 10: 多级缓存系统

9 实战练习与案例分析

9.1 综合案例：智能文档问答系统

让我们通过一个完整的案例来整合所有学到的技术。

```
1 import asyncio
2 import logging
3 from typing import Dict, List, Any
4 from dataclasses import dataclass
5 from pathlib import Path
6
7 # 配置日志
8 logging.basicConfig(level=logging.INFO)
9 logger = logging.getLogger(__name__)
10
11 @dataclass
12 class SystemConfig:
13     """系统配置"""
14     # Qdrant 配置
15     qdrant_host: str = "localhost"
16     qdrant_port: int = 6333
17
18     # OpenAI 配置
19     openai_api_key: str = "your-api-key"
20
21     # 向量模型配置
22     embedding_model: str = "all-MiniLM-L6-v2"
23     cross_encoder_model: str = "cross-encoder/ms-marco-MiniLM-L-6-v2"
24
25     # 性能配置
26     chunk_size: int = 512
27     chunk_overlap: int = 50
28     max_context_tokens: int = 3000
29     retrieval_top_k: int = 20
30     final_top_k: int = 5
31
32     # 缓存配置
33     enable_cache: bool = True
34     cache_ttl: int = 3600
35
36 class IntelligentDocumentQA:
37     """智能文档问答系统"""
38
39     def __init__(self, config: SystemConfig):
40         self.config = config
41
42     # 初始化组件
43     def _setup_components(self):
44
45     # 性能监控
46     self.monitor = PerformanceMonitor()
47
48     # 系统状态
49     self.is_ready = False
50
51     def _setup_components(self):
```



```
52     """初始化系统组件"""
53     logger.info("初始化系统组件...")
54
55     # 文档处理器
56     self.doc_processor = SemanticDocumentProcessor(
57         chunk_size=self.config.chunk_size,
58         overlap=self.config.chunk_overlap
59     )
60
61     # 向量检索器
62     self.retriever = HybridRetriever(
63         embedding_model=self.config.embedding_model,
64         cross_encoder_model=self.config.cross_encoder_model
65     )
66
67     # 缓存系统
68     if self.config.enable_cache:
69         try:
70             redis_client = redis.Redis(
71                 host='localhost',
72                 port=6379,
73                 decode_responses=False
74             )
75             redis_client.ping() # 测试连接
76
77             cache_strategy = AdaptiveCacheStrategy(base_ttl=self.
config.cache_ttl)
78             self.cache = MultiLevelCache(
79                 ll_size=1000,
80                 redis_client=redis_client,
81                 strategy=cache_strategy
82             )
83             logger.info("缓存系统初始化成功")
84         except Exception as e:
85             logger.warning(f"Redis连接失败，禁用缓存: {e}")
86             self.config.enable_cache = False
87
88     # RAG系统
89     self.rag_system = AdvancedRAGSystem(self.config.openai_api_key)
90
91     logger.info("系统组件初始化完成")
92
93     async def load_documents_from_directory(self, directory_path: str)
-> int:
94         """从目录加载文档"""
95         directory = Path(directory_path)
96         if not directory.exists():
97             raise ValueError(f"目录不存在: {directory_path}")
98
99         supported_extensions = {'.txt', '.md', '.pdf', '.docx'}
100         document_files = []
101
102         for ext in supported_extensions:
103             document_files.extend(directory.glob(f"*{ext}"))
104
105         logger.info(f"发现 {len(document_files)} 个文档文件")
106
```

```
107     loaded_count = 0
108     documents = []
109     metadatas = []
110
111     for file_path in document_files:
112         try:
113             # 根据文件类型读取内容
114             content = await self._read_document_file(file_path)
115             if content.strip():
116                 documents.append(content)
117                 metadatas.append({
118                     'filename': file_path.name,
119                     'filepath': str(file_path),
120                     'file_size': file_path.stat().st_size,
121                     'file_type': file_path.suffix
122                 })
123                 loaded_count += 1
124
125         except Exception as e:
126             logger.error(f"读取文件失败 {file_path}: {e}")
127
128     if documents:
129         # 添加到知识库
130         await self._add_documents_async(documents, metadatas)
131
132     logger.info(f"成功加载 {loaded_count} 个文档")
133     return loaded_count
134
135     async def _read_document_file(self, file_path: Path) -> str:
136         """读取文档文件内容"""
137         if file_path.suffix == '.txt' or file_path.suffix == '.md':
138             with open(file_path, 'r', encoding='utf-8') as f:
139                 return f.read()
140
141         elif file_path.suffix == '.pdf':
142             # 简化的PDF读取 (需要安装PyPDF2)
143             try:
144                 import PyPDF2
145                 with open(file_path, 'rb') as f:
146                     reader = PyPDF2.PdfReader(f)
147                     text = ""
148                     for page in reader.pages:
149                         text += page.extract_text() + "\n"
150                     return text
151             except ImportError:
152                 logger.error("需要安装PyPDF2来处理PDF文件")
153                 return ""
154
155         elif file_path.suffix == '.docx':
156             # 简化的DOCX读取 (需要安装python-docx)
157             try:
158                 import docx
159                 doc = docx.Document(file_path)
160                 text = ""
161                 for paragraph in doc.paragraphs:
162                     text += paragraph.text + "\n"
163                 return text
```

```
164         except ImportError:
165             logger.error("需要安装python-docx来处理DOCX文件")
166             return ""
167
168         return ""
169
170     async def _add_documents_async(self, documents: List[str],
171     metadatas: List[Dict]):
172         """异步添加文档到知识库"""
173         # 这里可以使用异步处理来提高性能
174         self.rag_system.add_documents_to_knowledge_base(documents,
175         metadatas)
176         self.is_ready = True
177
178     async def answer_question(self, question: str) -> Dict[str, Any]:
179         """回答用户问题"""
180         if not self.is_ready:
181             return {
182                 "error": "系统尚未准备就绪，请先加载文档",
183                 "answer": None,
184                 "sources": [],
185                 "performance": {}
186             }
187
188         start_time = time.time()
189
190         try:
191             # 检查缓存
192             if self.config.enable_cache:
193                 cache_key = hashlib.md5(question.encode()).hexdigest()
194                 cached_result = self.cache.get('qa_result', {'question': cache_key})
195                 if cached_result:
196                     logger.info("从缓存返回答案")
197                     return cached_result
198
199             # 执行RAG查询
200             result = self.rag_system.query(
201                 question=question,
202                 retrieval_top_k=self.config.retrieval_top_k,
203                 final_top_k=self.config.final_top_k,
204                 use_reranking=True
205             )
206
207             # 计算性能指标
208             latency_ms = (time.time() - start_time) * 1000
209             self.monitor.record_query_latency(latency_ms)
210
211             # 添加性能信息
212             result['performance'] = {
213                 'latency_ms': latency_ms,
214                 'retrieval_count': len(result['sources']),
215                 'from_cache': False
216             }
217
218             # 存储到缓存
219             if self.config.enable_cache:
```

```

218         self.cache.set('qa_result', {'question': cache_key},
219                             result, latency_ms/1000)
220
221         return result
222
223     except Exception as e:
224         logger.error(f"回答问题时出错: {e}")
225         self.monitor.record_error("query_error")
226
227         return {
228             "error": str(e),
229             "answer": "抱歉, 处理您的问题出现了错误。",
230             "sources": [],
231             "performance": {"latency_ms": (time.time() - start_time
232 ) * 1000}
233         }
234
235     def get_system_status(self) -> Dict[str, Any]:
236         """获取系统状态"""
237         status = {
238             "ready": self.is_ready,
239             "document_count": len(self.retriever.documents) if hasattr(
240 self.retriever, 'documents') else 0,
241             "performance": self.monitor.get_performance_report()
242         }
243
244         if self.config.enable_cache:
245             status["cache"] = self.cache.get_stats()
246
247         return status
248
249     async def health_check(self) -> bool:
250         """健康检查"""
251         try:
252             # 检查基础组件
253             if not self.is_ready:
254                 return False
255
256             # 测试简单查询
257             test_result = await self.answer_question("系统测试查询")
258             return "error" not in test_result
259
260         except Exception as e:
261             logger.error(f"健康检查失败: {e}")
262             return False
263
264 # 使用示例和演示
265 async def demo_intelligent_document_qa():
266     """演示智能文档问答系统"""
267
268     print("=== 智能文档问答系统演示 ===\n")
269
270     # 1. 系统配置
271     config = SystemConfig(
272         openai_api_key="your-actual-api-key", # 需要真实的API密钥
273         chunk_size=400,
274         retrieval_top_k=15,

```

```
272         final_top_k=4
273     )
274
275     # 2. 初始化系统
276     print("1. 初始化智能问答系统...")
277     qa_system = IntelligentDocumentQA(config)
278
279     # 3. 创建示例文档
280     print("2. 创建示例知识库...")
281
282     sample_docs = [
283         """
284         人工智能发展史
285
286         人工智能的概念最早可以追溯到1950年，当时艾伦·图灵提出了著名的"
287         图灵测试"来判断机器是否具有智能。
288
289         1956年，在达特茅斯学院举行的夏季研讨会上，"人工智能"这个术语被
290         正式提出，标志着AI作为一个学科的诞生。
291
292         1960年代到1970年代是AI的第一个黄金时期，专家系统得到了广泛的研
293         究和应用。然而，由于计算能力的限制和对AI复杂性的低估，AI在1970年代末
294         期进入了第一个"冬天"。
295
296         1980年代，随着个人计算机的普及和更好的算法，AI重新获得关注。机
297         器学习开始成为AI的重要分支。
298
299         进入21世纪，随着大数据、云计算和GPU计算能力的发展，深度学习技术
300         取得突破性进展，推动AI进入新的发展阶段。
301         """,
302         """
303         机器学习基础概念
304
305         机器学习是人工智能的一个分支，它使计算机系统能够通过经验自动改
306         进性能，而无需明确编程。
307
308         监督学习使用标记的训练数据来学习输入和输出之间的映射关系。常见
309         的监督学习任务包括分类和回归。
310         分类任务的目标是预测离散的类别标签，而回归任务的目标是预测连续
311         的数值。
312
313         无监督学习处理没有标签的数据，目标是发现数据中的隐藏模式。聚类
314         和降维是两种主要的无监督学习任务。
315
316         强化学习通过与环境交互来学习最优行为策略。代理通过试错和奖励信
317         号来学习如何在特定环境中最大化累积奖励。
318
319         深度学习是机器学习的一个子领域，它使用多层神经网络来学习数据的
320         复杂表示。深度学习在图像识别、语音识别和自然语言处理等领域取得了显著
321         成就。
322         """,
323         """
324         现代AI应用领域
```

计算机视觉是AI的重要应用领域之一。现代计算机视觉系统可以进行图像分类、目标检测、语义分割等任务。

在自动驾驶、医疗影像分析、安防监控等领域有广泛应用。

自然语言处理(NLP)使计算机能够理解和生成人类语言。现代NLP技术包括机器翻译、文本摘要、情感分析、问答系统等。

大型语言模型如GPT系列已经在很多NLP任务上达到了接近人类的性能水平。

推荐系统通过分析用户行为和偏好，为用户推荐可能感兴趣的内容或商品。协同过滤、内容过滤和深度学习是推荐系统的主要技术。

语音识别技术将语音信号转换为文本，语音合成技术则将文本转换为自然的语音。这些技术在智能助手、语音翻译等应用中发挥重要作用。

机器人技术结合了AI、控制系统和机械工程，使机器人能够在复杂环境中自主工作。服务机器人、工业机器人等正在改变人们的工作和生活方式。

```
"""
]

metadatas = [
    {"title": "人工智能发展史", "category": "历史", "difficulty": "初级"},
    {"title": "机器学习基础", "category": "技术", "difficulty": "中级"},
    {"title": "AI应用领域", "category": "应用", "difficulty": "中级"}
]

# 添加文档到系统
qa_system.rag_system.add_documents_to_knowledge_base(sample_docs, metadatas)
qa_system.is_ready = True

# 4. 测试问答功能
print("3. 测试智能问答功能...\n")

test_questions = [
    "人工智能这个术语是什么时候提出的？",
    "什么是监督学习？它和无监督学习有什么区别？",
    "深度学习在哪些应用领域取得了成功？",
    "推荐系统的主要技术有哪些？",
    "图灵测试是用来做什么的？"
]

for i, question in enumerate(test_questions, 1):
    print(f"=== 问题 {i} ===")
    print(f"用户问题: {question}")

# 获取答案
result = await qa_system.answer_question(question)

if "error" in result:
    print(f"错误: {result['error']}")
```

```

359         else:
360             print(f"系统回答: {result['answer']}")
361             print(f"响应时间: {result['performance']['latency_ms']:.1f}
ms")
362             print(f"引用源数量: {len(result['sources'])}")
363
364             # 显示引用源
365             if result['sources']:
366                 print("引用来源:")
367                 for j, source in enumerate(result['sources'][:2]): #
只显示前2个
368                     print(f"  {j+1}. [{source['score']:.3f}] {source['
content'][:100]}...")
369
370             print("-" * 60 + "\n")
371
372             # 5. 系统状态检查
373             print("4. 系统状态检查...")
374             status = qa_system.get_system_status()
375             print(f"系统就绪: {status['ready']}")
376             print(f"文档数量: {status['document_count']}")
377
378             if 'performance' in status and 'query_performance' in status['
performance']:
379                 perf = status['performance']['query_performance']
380                 print(f"处理查询数: {perf['total_queries']}")
381                 print(f"平均响应时间: {perf['avg_latency_ms']:.1f}ms")
382
383             # 6. 健康检查
384             print("\n5. 系统健康检查...")
385             is_healthy = await qa_system.health_check()
386             print(f"系统健康状态: {'正常' if is_healthy else '异常'}")
387
388             print("\n=== 演示完成 ===")
389
390             # 运行演示
391             if __name__ == "__main__":
392                 asyncio.run(demo_intelligent_document_qa())

```

Listing 11: 智能文档问答系统完整实现

10 总结与进阶学习路径

10.1 技术要点回顾

通过本教学指南，我们深入学习了 AI/LLM 专业技术和数据向量技术的核心内容：

1. **大语言模型基础**: 理解 Transformer 架构、注意力机制的数学原理
2. **提示工程**: 掌握零样本、少样本、思维链等提示技术
3. **上下文管理**: 学会 token 计算、上下文优化等关键技能
4. **MCP 协议**: 深入理解模型上下文协议的实现和应用

- 5. **多智能体系统:** 掌握 AutoGen、CrewAI 等框架的协作机制
- 6. **向量数据库:** 深入了解 HNSW 算法、Qdrant 优化配置
- 7. **RAG 系统:** 构建完整的检索增强生成系统
- 8. **性能优化:** 学会缓存策略、性能监控等工程实践

10.2 进阶学习建议

Table 4: 进阶学习路径规划

学习阶段	重点内容	实践项目	预期时间
基础巩固	深入理解本文档所有概念和代码	复现所有示例代码	2-3 周
技术深化	学习更多向量算法、优化技术	构建大规模向量搜索系统	1 个月
工程实践	分布式系统、微服务架构	部署生产级 RAG 系统	1-2 个月
前沿探索	最新研究论文、新兴技术	参与开源项目贡献	持续学习

10.3 推荐资源

技术文档和 API 参考

- Qdrant 官方文档: <https://qdrant.tech/documentation/>
- OpenAI API 参考: <https://platform.openai.com/docs>
- Hugging Face Transformers: <https://huggingface.co/docs/transformers>

开源项目学习

- LangChain: 构建 LLM 应用的框架
- LlamaIndex: 数据框架用于 LLM 应用
- AutoGen: 多智能体对话框架
- CrewAI: 任务导向的 AI 智能体协作
- Sentence Transformers: 语义相似度和检索

学术论文阅读

- "Attention Is All You Need" - Transformer 原理论文
- "Retrieval-Augmented Generation" - RAG 系统基础
- "Efficient and Robust Approximate Nearest Neighbor Search" - HNSW 算法
- "In-Context Learning and Induction Heads" - 上下文学习机制

11 附录：常用公式和算法

11.1 数学公式速查

11.1.1 向量相似度计算

$$\text{余弦相似度: } \cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}| |\mathbf{b}|} \quad (13)$$

$$\text{欧氏距离: } d(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2} \quad (14)$$

$$\text{曼哈顿距离: } d(\mathbf{a}, \mathbf{b}) = \sum_{i=1}^n |a_i - b_i| \quad (15)$$

$$\text{内积相似度: } \mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i \quad (16)$$

11.1.2 注意力机制

$$\text{缩放点积注意力: } \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (17)$$

$$\text{多头注意力: } \text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (18)$$

$$\text{其中: } \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (19)$$

11.2 性能评估指标

11.2.1 检索质量指标

$$\text{准确率: } P = \frac{|\text{检索相关} \cap \text{实际相关}|}{|\text{检索相关}|} \quad (20)$$

$$\text{召回率: } R = \frac{|\text{检索相关} \cap \text{实际相关}|}{|\text{实际相关}|} \quad (21)$$

$$\text{F1 分数: } F1 = 2 \times \frac{P \times R}{P + R} \quad (22)$$

$$\text{平均准确率: } \text{mAP} = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \text{AP}(q) \quad (23)$$

11.2.2 系统性能指标

$$\text{平均响应时间: } \bar{t} = \frac{1}{n} \sum_{i=1}^n t_i \quad (24)$$

$$\text{吞吐量: } \text{QPS} = \frac{\text{总请求数}}{\text{总时间}} \quad (25)$$

$$\text{百分位延迟: } P_{95} = \text{第 95 百分位的响应时间} \quad (26)$$

$$\text{可用性: } \text{Availability} = \frac{\text{正常运行时间}}{\text{总时间}} \times 100\% \quad (27)$$

11.3 算法复杂度分析

Table 5: 常用算法时间复杂度对比

算法	构建复杂度	查询复杂度	空间复杂度
线性搜索	$O(n)$	$O(n \cdot d)$	$O(n \cdot d)$
LSH	$O(n \cdot d \cdot L)$	$O(L \cdot d)$	$O(n \cdot L)$
HNSW	$O(n \cdot \log n \cdot d)$	$O(\log n \cdot d)$	$O(n \cdot M)$
IVF	$O(n \cdot d + k \cdot d^2)$	$O(n_{probe} \cdot d)$	$O(n \cdot d)$

其中:

- n : 数据点数量
- d : 向量维度
- L : LSH 中哈希表数量
- M : HNSW 中每个节点的连接数
- k : IVF 中聚类中心数量
- n_{probe} : IVF 中探测的聚类数量

12 实用工具和脚本

12.1 性能测试脚本

```

1 #!/usr/bin/env python3
2 """
3 向量搜索性能测试工具
4 用于评估不同配置下的搜索性能
5 """
6
7 import time
8 import numpy as np
9 import matplotlib.pyplot as plt
10 from qdrant_client import QdrantClient

```

```
11 from qdrant_client.http import models
12 from sentence_transformers import SentenceTransformer
13 from typing import List, Dict, Tuple
14 import json
15
16 class VectorSearchBenchmark:
17     """向量搜索基准测试"""
18
19     def __init__(self, client: QdrantClient, collection_name: str):
20         self.client = client
21         self.collection_name = collection_name
22         self.encoder = SentenceTransformer('all-MiniLM-L6-v2')
23
24     def generate_test_data(self, num_docs: int = 10000) -> Tuple[List[
25         str], List[np.ndarray]]:
26         """生成测试数据"""
27         print(f"生成 {num_docs} 个测试文档...")
28
29         # 生成模拟文档
30         topics = ["科技", "教育", "医疗", "金融", "娱乐", "体育", "新闻", "生活"]
31         documents = []
32
33         for i in range(num_docs):
34             topic = topics[i % len(topics)]
35             doc = f"{topic}相关的文档内容 {i}, 包含一些测试数据和信息。"
36             documents.append(doc)
37
38         # 生成向量
39         print("编码文档向量...")
40         vectors = self.encoder.encode(documents, show_progress_bar=True)
41
42         return documents, vectors
43
44     def create_test_collection(self, num_docs: int = 10000):
45         """创建测试集合"""
46         try:
47             self.client.delete_collection(self.collection_name)
48         except:
49             pass
50
51         # 生成测试数据
52         documents, vectors = self.generate_test_data(num_docs)
53
54         # 创建集合
55         self.client.create_collection(
56             collection_name=self.collection_name,
57             vectors_config=models.VectorParams(
58                 size=vectors.shape[1],
59                 distance=models.Distance.COSINE
60             )
61         )
62
63         # 批量插入
64         points = []
```

```

64         for i, (doc, vector) in enumerate(zip(documents, vectors)):
65             points.append(models.PointStruct(
66                 id=i,
67                 vector=vector.tolist(),
68                 payload={"text": doc, "doc_id": i}
69             ))
70
71     # 分批上传
72     batch_size = 1000
73     for i in range(0, len(points), batch_size):
74         batch = points[i:i + batch_size]
75         self.client.upsert(
76             collection_name=self.collection_name,
77             points=batch
78         )
79         if (i // batch_size + 1) % 10 == 0:
80             print(f"已上传 {i + len(batch)} / {len(points)} 个点")
81
82     print(f"测试集合创建完成, 包含 {len(points)} 个向量")
83
84     def benchmark_search_latency(self, queries: List[str], top_k: int =
85     10) -> Dict:
86         """测试搜索延迟"""
87         print(f"测试搜索延迟, 查询数量: {len(queries)}")
88
89         latencies = []
90
91         for query in queries:
92             query_vector = self.encoder.encode([query])[0].tolist()
93
94             start_time = time.time()
95
96             results = self.client.search(
97                 collection_name=self.collection_name,
98                 query_vector=query_vector,
99                 limit=top_k
100             )
101
102             latency = (time.time() - start_time) * 1000 # 转换为毫秒
103             latencies.append(latency)
104
105         return {
106             "avg_latency_ms": np.mean(latencies),
107             "p50_latency_ms": np.percentile(latencies, 50),
108             "p95_latency_ms": np.percentile(latencies, 95),
109             "p99_latency_ms": np.percentile(latencies, 99),
110             "min_latency_ms": np.min(latencies),
111             "max_latency_ms": np.max(latencies),
112             "qps_estimate": 1000 / np.mean(latencies),
113             "latencies": latencies
114         }
115
116     def benchmark_different_configs(self, test_queries: List[str]) ->
117     Dict:
118         """测试不同配置的性能"""
119         configs = [
120             {"m": 16, "ef_construct": 100, "ef_search": 50},

```

```

119         {"m": 16, "ef_construct": 200, "ef_search": 100},
120         {"m": 32, "ef_construct": 100, "ef_search": 50},
121         {"m": 32, "ef_construct": 200, "ef_search": 100},
122     ]
123
124     results = {}
125
126     for i, config in enumerate(configs):
127         config_name = f"M{config['m']}_EFC{config['ef_construct']}_EFS{config['ef_search']}"
128         print(f"\n测试配置 {i+1}/{len(configs)}: {config_name}")
129
130         # 更新HNSW配置
131         try:
132             self.client.update_collection(
133                 collection_name=self.collection_name,
134                 hnsw_config=models.HnswConfig(
135                     m=config["m"],
136                     ef_construct=config["ef_construct"]
137                 )
138             )
139
140             # 等待索引重建
141             time.sleep(2)
142
143             # 测试性能
144             perf_results = self.benchmark_search_latency(
145                 test_queries)
146             results[config_name] = {
147                 "config": config,
148                 "performance": perf_results
149             }
150
151         except Exception as e:
152             print(f"配置 {config_name} 测试失败: {e}")
153
154     return results
155
156     def generate_performance_report(self, results: Dict, output_file:
157     str = "benchmark_report.json"):
158         """生成性能报告"""
159
160         # 保存详细结果
161         with open(output_file, 'w', encoding='utf-8') as f:
162             json.dump(results, f, indent=2, ensure_ascii=False)
163
164         # 生成可视化图表
165         self.plot_performance_comparison(results)
166
167         # 打印摘要
168         print("\n=== 性能测试摘要 ===")
169         for config_name, result in results.items():
170             perf = result["performance"]
171             print(f"\n{config_name}:")
172             print(f"    平均延迟: {perf['avg_latency_ms']:.1f}ms")
173             print(f"    P95延迟: {perf['p95_latency_ms']:.1f}ms")
174             print(f"    估算QPS: {perf['qps_estimate']:.1f}")

```

```

173     def plot_performance_comparison(self, results: Dict):
174         """绘制性能对比图"""
175         configs = list(results.keys())
176         avg_latencies = [results[c]["performance"]["avg_latency_ms"]
177         for c in configs]
177         p95_latencies = [results[c]["performance"]["p95_latency_ms"]
178         for c in configs]
178         qps_estimates = [results[c]["performance"]["qps_estimate"] for
179         c in configs]
179
180         fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))
181
182         # 延迟对比
183         x = np.arange(len(configs))
184         width = 0.35
185
186         ax1.bar(x - width/2, avg_latencies, width, label='平均延迟',
187         alpha=0.8)
187         ax1.bar(x + width/2, p95_latencies, width, label='P95延迟',
188         alpha=0.8)
188
189         ax1.set_xlabel('配置')
190         ax1.set_ylabel('延迟 (ms)')
191         ax1.set_title('搜索延迟对比')
192         ax1.set_xticks(x)
193         ax1.set_xticklabels(configs, rotation=45, ha='right')
194         ax1.legend()
195         ax1.grid(True, alpha=0.3)
196
197         # QPS对比
198         ax2.bar(configs, qps_estimates, color='green', alpha=0.7)
199         ax2.set_xlabel('配置')
200         ax2.set_ylabel('QPS')
201         ax2.set_title('吞吐量对比')
202         ax2.tick_params(axis='x', rotation=45)
203         ax2.grid(True, alpha=0.3)
204
205         plt.tight_layout()
206         plt.savefig('performance_comparison.png', dpi=300, bbox_inches=
207         'tight')
207         plt.show()
208
209         print("性能对比图已保存为 'performance_comparison.png'")
210
211 # 使用示例
212 def run_benchmark():
213     """运行基准测试"""
214     client = QdrantClient("localhost", port=6333)
215     benchmark = VectorSearchBenchmark(client, "benchmark_test")
216
217     # 创建测试数据
218     benchmark.create_test_collection(num_docs=5000)
219
220     # 准备测试查询
221     test_queries = [
222         "科技发展趋势",
223         "教育改革政策",

```

```

224     "医疗健康",
225     "金融投资策略",
226     "体育赛事分析",
227     "新闻热点事件",
228     "生活服务指南",
229     "娱乐内容推荐"
230 ] * 10 # 重复以增加测试数量
231
232 # 执行基准测试
233 results = benchmark.benchmark_different_configs(test_queries)
234
235 # 生成报告
236 benchmark.generate_performance_report(results)
237
238 if __name__ == "__main__":
239     run_benchmark()

```

Listing 12: 向量搜索性能测试工具

12.2 系统监控仪表板

```

1  #!/usr/bin/env python3
2  """
3  AI/LLM系统实时性能监控仪表板
4  使用Streamlit构建Web界面
5  """
6
7  import streamlit as st
8  import plotly.graph_objects as go
9  import plotly.express as px
10 from plotly.subplots import make_subplots
11 import pandas as pd
12 import time
13 import json
14 from datetime import datetime, timedelta
15 import redis
16 from typing import Dict, List
17
18 class AISystemMonitor:
19     """AI系统监控器"""
20
21     def __init__(self):
22         # 连接Redis用于存储监控数据
23         try:
24             self.redis_client = redis.Redis(host='localhost', port
25             =6379, decode_responses=True)
26             self.redis_client.ping()
27             self.redis_available = True
28         except:
29             self.redis_available = False
30             st.warning("Redis连接失败, 使用内存存储 (数据不会持久化)")
31             self.memory_store = {}
32
33     def get_system_metrics(self) -> Dict:
34         """获取系统指标"""
35         if self.redis_available:

```

```
35         try:
36             metrics_str = self.redis_client.get("system_metrics")
37             if metrics_str:
38                 return json.loads(metrics_str)
39         except:
40             pass
41
42     # 返回模拟数据
43     import random
44     import numpy as np
45
46     current_time = datetime.now()
47     base_latency = 150 + random.gauss(0, 30)
48     base_qps = 85 + random.gauss(0, 15)
49
50     return {
51         "timestamp": current_time.isoformat(),
52         "query_latency_ms": max(10, base_latency),
53         "qps": max(0, base_qps),
54         "cache_hit_rate": 0.75 + random.gauss(0, 0.1),
55         "memory_usage_mb": 1200 + random.randint(-100, 200),
56         "cpu_usage_percent": 45 + random.randint(-15, 25),
57         "active_connections": random.randint(10, 50),
58         "error_rate": max(0, random.gauss(0.02, 0.01)),
59         "vector_db_size": 1250000 + random.randint(-1000, 1000),
60         "embedding_queue_size": random.randint(0, 20)
61     }
62
63     def get_historical_data(self, hours: int = 24) -> List[Dict]:
64         """获取历史数据"""
65         data = []
66         current_time = datetime.now()
67
68         for i in range(hours * 12): # 每5分钟一个数据点
69             timestamp = current_time - timedelta(minutes=i*5)
70
71             # 生成模拟历史数据
72             import random
73
74             base_latency = 150 + random.gauss(0, 20) + 10 * np.sin(i *
75 0.1)
76             base_qps = 80 + random.gauss(0, 10) + 20 * np.sin(i * 0.08
77 + 1)
78
79             data.append({
80                 "timestamp": timestamp,
81                 "query_latency_ms": max(10, base_latency),
82                 "qps": max(0, base_qps),
83                 "cache_hit_rate": min(1.0, max(0, 0.7 + random.gauss(0,
84 0.1))),
85                 "memory_usage_mb": 1200 + random.randint(-50, 100),
86                 "cpu_usage_percent": max(0, min(100, 50 + random.
87 randint(-20, 30))),
88                 "error_rate": max(0, random.gauss(0.015, 0.005))
89             })
90
91         return list(reversed(data))
```



```

89 def create_realtime_dashboard():
90     """创建实时监控仪表板"""
91     st.set_page_config(
92         page_title="AI/LLM系统监控",
93         page_icon="📊",
94         layout="wide"
95     )
96
97     st.title("AI/LLM系统实时监控仪表板")
98     st.markdown("----")
99
100     # 初始化监控器
101     monitor = AISystemMonitor()
102
103     # 侧边栏控制
104     st.sidebar.header("监控设置")
105     auto_refresh = st.sidebar.checkbox("自动刷新", value=True)
106     refresh_interval = st.sidebar.slider("刷新间隔(秒)", 1, 30, 5)
107     show_details = st.sidebar.checkbox("显示详细信息", value=False)
108
109     # 创建占位符
110     metrics_placeholder = st.empty()
111     charts_placeholder = st.empty()
112
113     # 主监控循环
114     while True:
115         current_metrics = monitor.get_system_metrics()
116         historical_data = monitor.get_historical_data()
117
118         with metrics_placeholder.container():
119             # 关键指标卡片
120             col1, col2, col3, col4 = st.columns(4)
121
122             with col1:
123                 st.metric(
124                     label="查询延迟",
125                     value=f"{current_metrics['query_latency_ms']:.1f}ms",
126                     delta=f"{random.uniform(-5, 5):.1f}ms"
127                 )
128
129                 st.metric(
130                     label="QPS",
131                     value=f"{current_metrics['qps']:.0f}",
132                     delta=f"{random.uniform(-3, 8):.0f}"
133                 )
134
135             with col2:
136                 cache_hit_rate = current_metrics['cache_hit_rate'] *
100
137                 st.metric(
138                     label="缓存命中率",
139                     value=f"{cache_hit_rate:.1f}%",
140                     delta=f"{random.uniform(-2, 3):.1f}%"
141                 )
142
143                 error_rate = current_metrics['error_rate'] * 100

```

```

144         st.metric(
145             label="    错误率",
146             value=f"{error_rate:.3f}%",
147             delta=f"{random.uniform(-0.01, 0.02):.3f}%"
148         )
149
150     with col3:
151         st.metric(
152             label="    内存使用",
153             value=f"{current_metrics['memory_usage_mb']:.0f}MB"
154             ,
155             delta=f"{random.uniform(-20, 50):.0f}MB"
156         )
157         st.metric(
158             label="    CPU使用率",
159             value=f"{current_metrics['cpu_usage_percent']:.1f}%"
160             ,
161             delta=f"{random.uniform(-5, 10):.1f}%"
162         )
163     with col4:
164         st.metric(
165             label="    活跃连接",
166             value=f"{current_metrics['active_connections']}",
167             delta=f"{random.randint(-3, 5)}"
168         )
169
170         st.metric(
171             label="    向量数据库大小",
172             value=f"{current_metrics['vector_db_size']:,}",
173             delta=f"{random.randint(-100, 500)}"
174         )
175
176     with charts_placeholder.container():
177         # 创建图表
178         df = pd.DataFrame(historical_data)
179
180         # 延迟和QPS趋势图
181         fig_performance = make_subplots(
182             rows=2, cols=2,
183             subplot_titles=('查询延迟趋势', 'QPS趋势', '缓存命中率',
184                             , 'CPU和内存使用'),
185             specs=[[{"secondary_y": False}, {"secondary_y": False
186                     }, {"secondary_y": False}, {"secondary_y": True}]]
187
188         # 延迟趋势
189         fig_performance.add_trace(
190             go.Scatter(
191                 x=df['timestamp'],
192                 y=df['query_latency_ms'],
193                 mode='lines',
194                 name='延迟(ms)',
195                 line=dict(color='blue', width=2)
196             ),

```

```
197         row=1, col=1
198     )
199
200     # QPS 趋势
201     fig_performance.add_trace(
202         go.Scatter(
203             x=df['timestamp'],
204             y=df['qps'],
205             mode='lines',
206             name='QPS',
207             line=dict(color='green', width=2)
208         ),
209         row=1, col=2
210     )
211
212     # 缓存命中率
213     fig_performance.add_trace(
214         go.Scatter(
215             x=df['timestamp'],
216             y=df['cache_hit_rate'] * 100,
217             mode='lines',
218             name='命中率(%)',
219             line=dict(color='orange', width=2),
220             fill='tonexty'
221         ),
222         row=2, col=1
223     )
224
225     # CPU和内存
226     fig_performance.add_trace(
227         go.Scatter(
228             x=df['timestamp'],
229             y=df['cpu_usage_percent'],
230             mode='lines',
231             name='CPU(%)',
232             line=dict(color='red', width=2)
233         ),
234         row=2, col=2
235     )
236
237     fig_performance.add_trace(
238         go.Scatter(
239             x=df['timestamp'],
240             y=df['memory_usage_mb'],
241             mode='lines',
242             name='内存(MB)',
243             line=dict(color='purple', width=2),
244             yaxis='y2'
245         ),
246         row=2, col=2, secondary_y=True
247     )
248
249     fig_performance.update_layout(
250         height=600,
251         title_text="系统性能监控",
252         showlegend=True
253     )
```

```

254         st.plotly_chart(fig_performance, use_container_width=True)
255
256     # 详细信息表格
257     if show_details:
258         st.subheader("    详细系统信息")
259
260         col1, col2 = st.columns(2)
261
262         with col1:
263             st.write("**当前指标**")
264             metrics_df = pd.DataFrame([current_metrics]).T
265             metrics_df.columns = ['数值']
266             st.dataframe(metrics_df)
267
268         with col2:
269             st.write("**最近24小时统计**")
270             stats = {
271                 "平均延迟(ms)": f"{df['query_latency_ms'].mean():.1f}",
272                 "最大延迟(ms)": f"{df['query_latency_ms'].max():.1f}",
273                 "平均QPS": f"{df['qps'].mean():.1f}",
274                 "平均缓存命中率": f"{df['cache_hit_rate'].mean():.1%}",
275                 "平均CPU使用率": f"{df['cpu_usage_percent'].mean():.1f}%",
276                 "平均内存使用": f"{df['memory_usage_mb'].mean():.0f}MB"
277             }
278             stats_df = pd.DataFrame(list(stats.items()),
279                                     columns=['指标', '数值'])
280             st.dataframe(stats_df, hide_index=True)
281
282     # 状态指示器
283     status_color = " " if current_metrics['error_rate'] < 0.05
284     else " " if current_metrics['error_rate'] < 0.1 else " "
285     st.sidebar.write(f"**系统状态**: {status_color}")
286     st.sidebar.write(f"**最后更新**: {datetime.now().strftime('%H:%M:%S')}")
287
288     if not auto_refresh:
289         break
290
291     time.sleep(refresh_interval)
292
293 if __name__ == "__main__":
294     create_realtime_dashboard()

```

Listing 13: 实时性能监控仪表板

13 结语

本教学文档全面介绍了 AI/LLM 专业技术和数据向量技术的核心概念、数学原理、工程实践和性能优化方法。通过理论学习、代码实践和案例分析，读者应该能够：

1. 深入理解大语言模型的工作原理和应用方法
2. 掌握提示工程和上下文管理的关键技能
3. 学会设计和实现多智能体协作系统
4. 熟练使用向量数据库进行高效检索
5. 构建完整的 RAG 系统解决实际问题
6. 进行系统性能优化和监控

随着 AI 技术的快速发展，这些基础知识将为进一步的学习和实践提供坚实的基础。建议读者在掌握本文档内容的基础上，持续关注最新的技术发展，参与开源项目，并在实际项目中应用所学知识。

记住：理论学习只是开始，实践应用才能真正掌握这些技术。祝愿每一位读者都能在 AI/LLM 领域取得成功！